

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования

**«Белорусский государственный университет информатики и
радиоэлектроники»**

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

ОТЧЕТ

по лабораторной работе №2

по дисциплине

**«Современные технологии программирования масштабируемых
приложений»**

«Депозитные операции с физическими лицами»

Выполнил:

Лебедев Артем Владимирович
магистрант группы 556301

Преподаватель:

Хмелёва Ангелина Владимировна
кандидат технических наук
доцент кафедры ПОИТ

Минск, 2026 г.

СОДЕРЖАНИЕ

1	Цель работы	1
2	Задание	2
3	Проектирование	3
3.1	Декомпозиция на микросервисы	4
3.2	План счетов и номер счёта	4
3.3	Схема данных	5
3.4	Схема проводок	6
3.5	Расчёт процентов	7
3.6	Заккрытие банковского дня	7
3.7	REST-интерфейсы	8
4	Реализация	9
4.1	Структура проекта	10
4.2	Главная книга	10
4.3	Заключение депозитного договора	11
4.4	Контроль корректности данных договора	12
4.5	Обработка банковского дня	12
5	Тестирование	13
6	Результаты работы	15
	Заключение	25
	Список использованных источников	26
	Приложение А Исходный код сервисов счетов и депозитов	27

1 ЦЕЛЬ РАБОТЫ

Реализовать микросервис депозитных операций с физическими лицами: справочник «План счетов», форму заключения депозитного договора с контролем вводимых данных, открытие счетов клиента по правилам банковского учёта, проводки по схеме «Депозитная программа», счёт фонда развития банка и процедуру «Закрытие банковского дня» с отчётом о состоянии счетов. Данные о клиенте берутся из модуля «Клиенты» первой лабораторной работы.

2 ЗАДАНИЕ

Вариант №4 соответствует банку «Решение» в списке банков Республики Беларусь (<https://select.by/banki>). Для него выбраны две депозитные программы для физических лиц (таблица 2.1); условия приближены к действующим предложениям банка.

Таблица 2.1 – Депозитные программы банка «Решение»

	«Свободное накопление»	«Online-решение»
Вид вклада	отзывный, до востребования	срочный безотзывный
Выплата процентов	ежемесячно	в конце срока
Досрочный отзыв	возможен в любой день	невозможен
Ставка, BYN	7,00 % годовых	12,90 % годовых
Ставка, USD	1,50 % годовых	3,00 % годовых
Минимальная сумма	100	500
Срок договора	1–12 месяцев	3–13 месяцев
Счёт основной суммы	3404	3414
Счёт процентов	3470	3471

Требования задания:

1. разработать справочник «План счетов» и внести в него счета, необходимые для обслуживания выбранных депозитных линий;
2. разработать форму заключения депозитного договора (вид депозита и валюта из справочников, номер договора, даты начала и окончания, срок, сумма, процент) с контролем корректности данных;
3. при заключении договора создавать не менее двух счетов — для основной суммы и для процентов — с 13-значным номером, кодом и активностью из плана счетов, дебетом, кредитом, сальдо и названием по ФИО клиента;
4. предусмотреть счёт фонда развития банка (СФРБ) со стартовым капиталом 1 млн белорусских рублей;
5. разработать процедуру «Закрытие банковского дня» и отчёт о состоянии счетов; при активных депозитных программах в нём должно быть не менее шести счетов.

3 ПРОЕКТИРОВАНИЕ

3.1 Декомпозиция на микросервисы

Депозитные операции затрагивают три разных области ответственности, и каждой соответствует свой микросервис со своей базой данных (рисунок 3.1):

- `client-service` — клиенты банка (лабораторная работа № 1);
- `account-service` — главная книга: план счетов, лицевые счета, проводки, банковский день;
- `deposit-service` — депозитные программы и договоры, расчёт процентов.

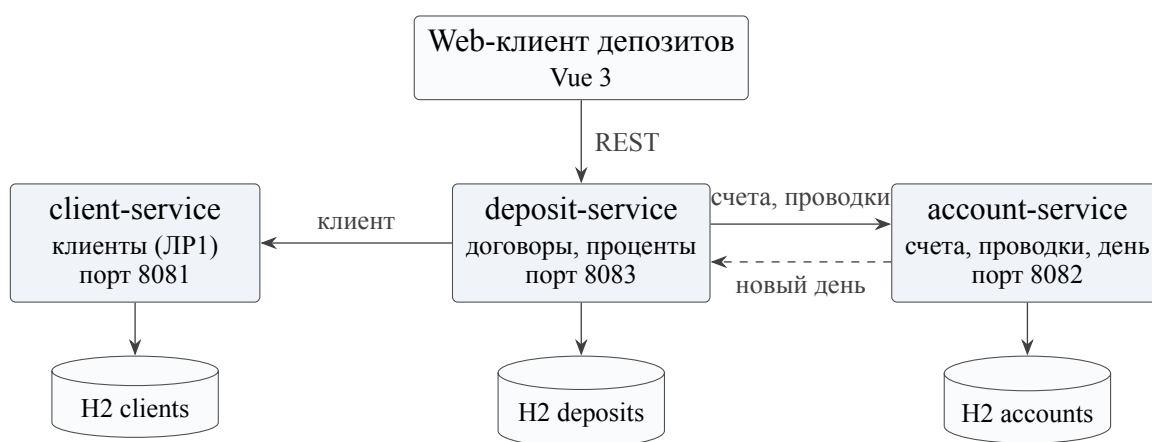


Рисунок 3.1 – Микросервисы депозитного контура и REST-вызовы между ними

Главная книга выделена в отдельный сервис намеренно. Со счетами работают и депозиты, и кредиты следующей лабораторной работы, а правила учёта — расчёт сальдо по активности счёта, запрет отрицательного остатка, неделимость проводки — должны соблюдаться в одном месте независимо от того, какой сервис инициировал операцию. Сервис депозитов не хранит ни оборотов, ни остатков: он знает только номера счетов своего договора и просит главную книгу провести операцию.

Сервисы общаются синхронно по REST. Клиенты смежных сервисов описаны декларативными интерфейсами Spring (@HttpExchange) в общей библиотеке `bank-common`; там же находится единый формат ошибки, который позволяет передать сообщение главной книги через сервис депозитов до web-клиента без искажений.

3.2 План счетов и номер счёта

Справочник «План счетов» (таблица 3.1) хранится в базе сервиса счетов и заполняется при старте. Активность балансового счёта определяет правило

расчёта сальдо: у активного счёта остаток равен дебету минус кредит, у пассивного — кредиту минус дебет.

Таблица 3.1 – Счета плана счетов, используемые депозитными программами

Код	Активность	Наименование	Назначение в работе
1010	активный	Денежные средства в кассе	касса банка
3404	пассивный	Вклады (депозиты) до востребования физических лиц	сумма отзывного вклада
3414	пассивный	Срочные вклады (депозиты) физических лиц	сумма срочного вклада
3470	пассивный	Начисленные процентные расходы по вкладам до востребования	проценты отзывного вклада
3471	пассивный	Начисленные процентные расходы по срочным вкладам	проценты срочного вклада
7327	пассивный	Фонд развития банка	СФРБ

Номер лицевого счёта состоит из 13 цифр:

$$\underbrace{b_1 b_2 b_3 b_4}_{\text{балансовый счёт}} \quad \underbrace{c_1 c_2 c_3 c_4 c_5}_{\text{код клиента}} \quad \underbrace{n_1 n_2 n_3}_{\text{номер счёта клиента}} \quad \underbrace{k}_{\text{ключ}}, \quad k = \left(3 \sum_{i=1}^{12} d_i w_i \right) \bmod 10, \quad (3.1)$$

где d_i — первые 12 цифр номера, w_i — веса, циклически повторяющие последовательность 7, 1, 3. Код клиента равен его идентификатору в модуле «Клиенты», у собственных счетов банка он нулевой. Веса 7, 1, 3 и множитель 3 взаимно просты с 10, поэтому ошибка в любой одной цифре номера меняет ключ и обнаруживается при проверке.

3.3 Схема данных

Базы сервисов независимы (рисунок 3.2). Договор ссылается на клиента и на счета не внешними ключами, а идентификатором клиента и номерами счетов: целостность таких ссылок обеспечивается проверкой через REST в момент заключения договора.

Таблица 3.2 – Проводки процесса «Депозитная программа»

№	Операция	Касса (А)		СФРБ (П)		Счёт вклада (П)		Проценты (П)	
		Дт	Кт	Дт	Кт	Дт	Кт	Дт	Кт
1	Внесение денег в кассу	+							
2	Перевод денег с кассы на текущий счёт		–				+		
3	Использование денег банком				+	–			
4	Начисление процентов по депозиту			–					+
5	Перевод процентов в кассу	+						–	
6	Вывод процентов из кассы		–						
7	Окончание депозита			–			+		
8	Перевод депозита в кассу	+				–			
9	Вывод денег из кассы		–						

Операции 1–3 выполняются при заключении договора, 4 — при каждом закрытии дня, 5–6 — в день выплаты процентов, 7–9 — по окончании срока или при досрочном отзыве.

3.5 Расчёт процентов

Проценты простые, начисляются за фактическое число дней при 365 днях в году. К дате t по договору на сумму S со ставкой r процентов годовых, заключённому в день t_0 , накоплено

$$I(t) = \text{round}\left(\frac{S \cdot r \cdot (t - t_0)}{100 \cdot 365}, 2\right), \quad (3.2)$$

а проводка очередного дня делается на разность $I(t)$ и уже начисленной суммы. Такой накопительный расчёт устойчивее суммирования дневных процентов: копейки округления не теряются, а пропущенный или повторно обработанный день не искажает итог. Например, вклад 10 000 BYN под 7 % за 31 день приносит $10000 \cdot 7 \cdot 31 / 36500 = 59,45$ BYN.

По отзывному вкладу накопленные проценты выплачиваются, когда истекает очередной полный месяц договора; по срочному — только в день окончания, вместе с возвратом суммы вклада.

3.6 Закрытие банковского дня

Банковский день хранится в сервисе счетов, но сам этот сервис не знает правил депозитов. Поэтому закрытие дня устроено как оповещение (рисунок 3.3): сервис счетов объявляет участникам дату нового дня, каждый участник начисляет проценты и проводит выплаты через обычный REST главной книги и возвращает протокол сделанного, после чего дата сдвигается.

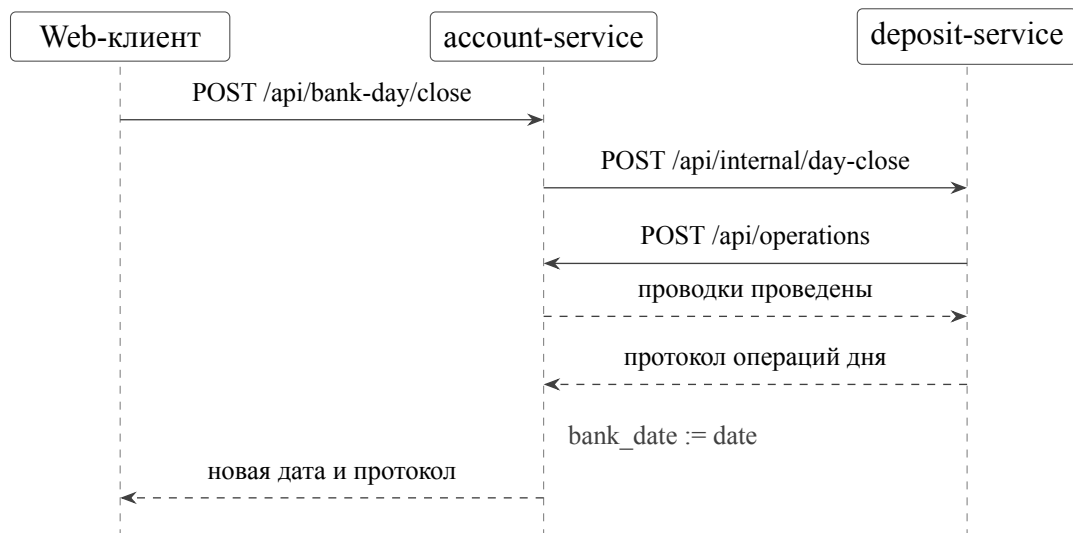


Рисунок 3.3 – Взаимодействие сервисов при закрытии банковского дня

Распределённая процедура должна переживать сбои, поэтому в неё заложены три свойства.

1. **Идемпотентность.** Каждый пакет проводок несёт ключ вида DEP-Д-000001-ACCRUE-2026-10-02. Главная книга запоминает ключи и на повторный запрос возвращает уже проведённые операции, не проводя их второй раз.
2. **Дата сдвигается последней.** Если участник ответил ошибкой, день остаётся открытым и закрытие можно повторить: уже обработанные договоры повторно не начисляются.
3. **Навёрстывание.** Недоступный участник не блокирует закрытие дня. Его расчёт накопительный (формула 3.2), поэтому при следующем закрытии он начислит проценты сразу за все пропущенные дни.

3.7 REST-интерфейсы

Операции двух сервисов приведены в таблице 3.3.

Таблица 3.3 – REST-интерфейсы сервисов счетов и депозитов

Метод	Адрес	Назначение
<i>account-service</i>		
GET	/api/chart-of-accounts, /api/currencies	справочники «План счетов» и валют
GET	/api/accounts?contractRef=	счета договора или клиента
GET	/api/accounts/system	счёт кассы или фонда развития в заданной валюте
POST	/api/accounts	открытие лицевого счёта
POST	/api/operations	атомарный пакет проводок с ключом идемпотентности
GET	/api/operations	журнал проводок
GET	/api/report	отчёт о состоянии счетов с итогами оборотов
GET	/api/bank-day	текущий банковский день
POST	/api/bank-day/close?days=	закрытие одного или нескольких дней
<i>deposit-service</i>		
GET	/api/products, /api/meta	виды депозитов, валюты, банковский день, номер договора
GET	/api/clients	список клиентов из модуля «Клиенты»
GET	/api/deposits, /api/deposits/{id}	договоры; договор со счетами и проводками
POST	/api/deposits	заключение договора
POST	/api/deposits/{id}/withdraw	досрочный отзыв вклада
POST	/api/internal/day-close	обработка нового банковского дня

4 РЕАЛИЗАЦИЯ

4.1 Структура проекта

К проекту первой работы добавлены три модуля Maven (листинг 4.1).

Листинг 4.1 – Модули лабораторной работы № 2

bank-common/	общая библиотека сервисов
api/	ApiError, BankException, обработчик ошибок REST
client/, ledger/	REST-клиенты ClientApi и LedgerApi, записи запросов и ответов
account-service/	главная книга
domain/	Account, ChartAccount, Operation, AccountNumbers, Activity
service/	LedgerService, BankDayService, DayCloseNotifier, LedgerInitializer
web/	LedgerController, BankDayController
resources/	schema.sql, data.sql (план счетов), static/ (отчёт по счетам)
deposit-service/	депозиты
domain/	DepositProduct, DepositContract, DepositKind
service/	DepositService, DepositPostings, InterestCalculator, DayCloseProcessor
web/	DepositController, ReferenceController, DayCloseController
resources/	schema.sql, data.sql (программы банка), static/ (форма договора)

4.2 Главная книга

Операция в главной книге — это набор строк «счёт, сторона, сумма». Сервис принимает не одиночную операцию, а пакет (листинг 4.2): заключение договора состоит из трёх операций схемы, и они должны быть проведены все вместе либо не проведены вовсе. Пакет выполняется в одной транзакции базы данных. Счета выбираются запросом `select ... for update`, чтобы два параллельных пакета не потеряли обновление оборотов. После каждой операции проверяется остаток затронутых счетов: отрицательное сальдо означает нехватку средств, и тогда исключение откатывает весь пакет вместе с записью о ключе.

Листинг 4.2 – Проведение пакета операций (LedgerService.post)

```
@Transactional
public List<OperationView> post(BatchRequest request) {
    if (batches.existsById(request.batchKey())) {           // повтор запроса: операции уже
        проведены
        return operations.findByBatchKeyOrderById(request.batchKey()).stream().map(
            OperationView::from).toList();
    }
    batches.saveAndFlush(new PostingBatch(request.batchKey()));

    LocalDate date = request.bankDate() != null ? request.bankDate() : bankDay.current();
    Map<String, Account> locked = new LinkedHashMap<>();
    List<Operation> posted = new ArrayList<>();
    for (OperationRequest item : request.operations()) {
        Operation operation = new Operation();
        ...
        Set<Account> touched = new LinkedHashSet<>();
        for (EntryRequest entry : item.entries()) {
```

```

        Account account = locked.computeIfAbsent(entry.account(), number -> accounts.
findByNumberForUpdate(number)
        .orElseThrow(() -> BankException.notFound("Счёт " + number + " не найден")
));
        ...
        account.apply(entry.side(), entry.amount());
        operation.addEntry(account, entry.side(), entry.amount());
        touched.add(account);
    }
    touched.forEach(account -> checkNotOverdrawn(account, item.description()));
    posted.add(operations.save(operation));
}
operations.flush();
return posted.stream().map(OperationView::from).toList();
}

```

При старте сервис открывает в каждой валюте кассу и фонд развития банка и зачисляет в фонд стартовый капитал. Размер капитала задаётся параметром `bank.fund.capital`; для этой работы сервис запускается со значением 1 000 000 BYN.

4.3 Заключение депозитного договора

Сервис депозитов проверяет условия договора, запрашивает клиента у модуля «Клиенты», открывает в главной книге два счёта и проводит операции 1–3 схемы (листинг 4.3). Балансовые счета берутся из вида вклада: 3404 и 3470 для вклада до востребования, 3414 и 3471 для срочного. Названием счёта служит ФИО клиента.

Листинг 4.3 – Открытие счетов и проводки приёма вклада (DepositService.open)

```

ClientInfo client = findClient(request.clientId());

DepositKind kind = product.getKind();
String number = request.number();
String currency = product.getCurrency();
String main = ledger.open(new OpenAccount(kind.getMainChartCode(), currency,
    client.id().intValue(), client.fullName(), number)).number();
String interest = ledger.open(new OpenAccount(kind.getInterestChartCode(), currency,
    client.id().intValue(), client.fullName(), number)).number();
post(key(number, "OPEN"), number, today,
    DepositPostings.open(cash(currency), fund(currency), main, request.amount()));

```

Сами проводки собраны в классе `DepositPostings`, который читается как таблица схемы (листинг 4.4).

Листинг 4.4 – Проводки приёма вклада и выплаты процентов (DepositPostings)

```

public static List<Posting> open(String cash, String fund, String main, BigDecimal amount) {
    return List.of(
        Posting.of("Внесение денег в кассу", Entry.debit(cash, amount)),
        Posting.of("Перевод денег с кассы на текущий счёт", Entry.credit(cash, amount),
            Entry.credit(main, amount)),
    );
}

```

```

        Posting.of("Использование денег банком", Entry.debit(main, amount), Entry.credit(
            fund, amount)));
    }

    public static Posting accrual(String fund, String interest, BigDecimal amount) {
        return Posting.of("Начисление процентов по депозиту", Entry.debit(fund, amount), Entry.
            credit(interest, amount));
    }

    public static List<Posting> payout(String cash, String interest, BigDecimal amount) {
        return List.of(
            Posting.of("Перевод процентов в кассу", Entry.debit(interest, amount), Entry.debit
                (cash, amount)),
            Posting.of("Вывод процентов из кассы", Entry.credit(cash, amount)));
    }
}

```

Открытие счёта в главной книге идемпотентно по паре «договор, балансовый счёт», а пакет проводок — по ключу DEP-<номер>-OPEN. Если сбой произойдёт после обращения к главной книге, но до сохранения договора, повтор запроса с тем же номером договора не создаст ни вторых счетов, ни вторых проводок.

4.4 Контроль корректности данных договора

Как и в первой работе, проверка двойная. Web-клиент проверяет форму до отправки, сервис повторяет проверки и добавляет те, что возможны только на сервере (таблица 4.1). Ставка и дата начала в форме недоступны для редактирования, но сервис всё равно сверяет их: запрос может прийти мимо формы или из устаревшей страницы.

Таблица 4.1 – Проверки формы депозитного договора

Поле	Правило	Где проверяется
Клиент	выбран; существует в модуле «Клиенты»	клиент, сервис
Номер договора	формат Д-000001; не занят	клиент, сервис
Вид депозита	запись справочника; доступен в выбранной валюте	клиент, сервис
Сумма	денежная; не меньше минимума программы	клиент, сервис
Срок	целое число месяцев в пределах программы	клиент, сервис
Процент	равен ставке программы	сервис
Дата начала	равна текущему банковскому дню	сервис
Дата окончания	равна дате начала плюс срок	сервис

4.5 Обработка банковского дня

Обработка договора на дату нового дня показана в листинге 4.5. Каждый договор обрабатывается в собственной транзакции, а ключи пакетов включают дату, поэтому повторный вызов для того же дня ничего не меняет.

Листинг 4.5 – Начисление, выплата и закрытие договора
(DepositService.processDay)

```

boolean matured = !date.isBefore(contract.getEndDate());
LocalDate upTo = matured ? contract.getEndDate() : date;

BigDecimal target = InterestCalculator.accrued(contract.getAmount(), contract.getRate(),
    contract.getStartDate(), upTo);
BigDecimal delta = target.subtract(contract.getAccrued());
if (delta.signum() > 0) {
    post(key(number, "ACCRUE-" + date), number, date,
        List.of(DepositPostings.accrual(fund(currency), contract.getInterestAccount(),
            delta)));
    contract.setAccrued(target);
}

int months = InterestCalculator.fullMonths(contract.getStartDate(), upTo);
boolean monthlyPayout = contract.getProduct().getKind() == DepositKind.REVOCABLE && months >
    contract.getPaidMonths();
if (matured || monthlyPayout) {
    payInterest(contract, date);
    contract.setPaidMonths(months);
}
if (matured) {
    close(contract, date);
}

```

5 ТЕСТИРОВАНИЕ

Сервисы покрыты 69 тестами (таблица 5.1). Чтобы тестировать сервис депозитов без запуска соседних микросервисов, смежные сервисы заменяются двойниками: клиент модуля «Клиенты» — заглушкой *Mockito*, а главная книга — классом *InMemoryLedger*, который реализует тот же интерфейс *LedgerApi* в памяти и соблюдает те же правила (сальдо по активности, запрет отрицательного остатка, идемпотентность). Это позволяет сверять в тестах обороты счетов с таблицей 3.2.

Таблица 5.1 – Тесты сервисов счетов и депозитов

Класс тестов	Тестов	Что проверяется
<i>account-service</i>		
<i>AccountNumbersTest</i>	7	структура номера, контрольный ключ, сальдо по активности
<i>LedgerServiceIT</i>	8	счета, обороты, атомарность пакета, идемпотентность
<i>BankDayIT</i>	6	закрытие дня, недоступный и отказавший участник, REST
<i>deposit-service</i>		
<i>InterestCalculatorTest</i>	13	формула процентов, полные месяцы
<i>DepositFlowIT</i>	7	проводки по схеме, выплаты, окончание и отзыв вклада
<i>DepositApiIT</i>	28	контроль данных договора, ошибки смежных сервисов
Всего	69	

Пример теста, который проходит весь срок срочного вклада и сверяет обороты со схемой, приведён в листинге 5.1; итог прогона — в листинге 5.2.

Листинг 5.1 – Тест срочного вклада (*DepositFlowIT*)

```
@Test
@DisplayName("Срочный вклад: проценты выплачиваются в конце срока вместе с возвратом суммы")
void irrevocableDepositPaysAtMaturity() {
    DepositView deposit = open("Д-000002", IRREVOCABLE_BYN, "5000.00", 3);

    closeDays(91); // 31.12: до окончания срока один
    // день
    assertThat(view(deposit).paid()).isEqualToComparingTo("0");
    assertThat(ledger.balance(deposit.interestAccount()).isEqualToComparingTo("160.81");

    List<String> events = closeDays(1); // 01.01.2027: окончание договора
    // , 92 дня
    DepositView closed = view(deposit);
    assertThat(closed.status()).isEqualTo("CLOSED");
    assertThat(closed.paid()).isEqualToComparingTo("162.58");

    // все счета клиента обнулились, касса транзитом пропустила вклад и проценты, фонд
    // потратил проценты
```

```
assertTurnover(deposit.mainAccount(), "10000.00", "10000.00", "0.00");
assertTurnover(deposit.interestAccount(), "162.58", "162.58", "0.00");
assertTurnover(cash, "10162.58", "10162.58", "0.00");
assertThat(ledger.balance(fund)).isEqualByComparingTo(InMemoryLedger.CAPITAL.subtract(new
BigDecimal("162.58")));
}
```

Листинг 5.2 – Результат выполнения тестов

```
$ ./mvnw -pl account-service,deposit-service -am verify

Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.066 s -- in by.bsuir.bank.account.AccountNumbersTest
Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.338 s -- in by.bsuir.bank.account.BankDayIT
Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.349 s -- in by.bsuir.bank.account.LedgerServiceIT
Tests run: 13, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.064 s -- in by.bsuir.bank.deposit.InterestCalculatorTest
Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.090 s -- in by.bsuir.bank.deposit.DepositFlowIT
Tests run: 28, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.565 s -- in by.bsuir.bank.deposit.DepositApiIT

Tests run: 69, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

6 РЕЗУЛЬТАТЫ РАБОТЫ

Сценарий демонстрации: два клиента открывают вклады по разным программам, банк работает три месяца, оба договора завершаются. Сервис счетов запущен со стартовым капиталом фонда развития 1 000 000 BYN.

После старта в отчёте два собственных счёта банка в белорусских рублях — касса и фонд развития (рисунок 6.1); справочник «План счетов» показан на рисунке 6.2.

Банк «Решение» учебная АБС

Клиенты

Счета

Депозиты

Кредиты

Банкомат

Счета банка

Отчёт о состоянии счетов и процедура «Закрытие банковского дня»

БАНКОВСКИЙ ДЕНЬ 01.10.2026

Закрыть день

30

Закрыть дней

ОТЧЁТ НА ДАТУ
01.10.2026

СЧЕТОВ В ОТЧЁТЕ
2

ФОНД РАЗВИТИЯ, BYN
1 000 000,00

КАССА, BYN
0,00

Отчёт по счетам

Журнал проводок

План счетов

BYN

USD

EUR

НОМЕР СЧЁТА	КОД	НАЗВАНИЕ СЧЁТА	АКТИВНОСТЬ	ДОГОВОР	ДЕБЕТ	КРЕДИТ	САЛДО
1010000000019	1010	Касса банка, BYN Денежные средства в кассе	Активный		0,00	0,00	0,00
7327000000029	7327	Фонд развития банка, BYN Фонд развития банка	Пассивный		0,00	1 000 000,00	1 000 000,00
Итого обороты, BYN					0,00	1 000 000,00	

Рисунок 6.1 – Отчёт по счетам после запуска: касса и фонд развития

Банк «Решение» учебная АБС

Клиенты

Счета

Депозиты

Кредиты

Банкомат

Счета банка

Отчёт о состоянии счетов и процедура «Закрытие банковского дня»

БАНКОВСКИЙ ДЕНЬ 01.10.2026

Закрыть день

30

Закрыть дней

ОТЧЁТ НА ДАТУ

01.10.2026

СЧЕТОВ В ОТЧЁТЕ

2

ФОНД РАЗВИТИЯ, BYN

1 000 000,00

КАССА, BYN

0,00

Отчёт по счетам

Журнал проводок

План счетов

БАЛАНСОВЫЙ СЧЁТ	НАИМЕНОВАНИЕ	АКТИВНОСТЬ
1010	Денежные средства в кассе	Активный
1201	Корреспондентский счёт в Национальном банке	Активный
2400	Кредиты физическим лицам	Активный
2470	Начисленные процентные доходы по кредитам физическим лицам	Активный
3012	Текущие (расчётные) счета коммерческих организаций	Пассивный
3014	Текущие (расчётные) счета физических лиц	Пассивный
3404	Вклады (депозиты) до востребования физических лиц	Пассивный
3414	Срочные вклады (депозиты) физических лиц	Пассивный
3470	Начисленные процентные расходы по вкладам (депозитам) до востребования	Пассивный
3471	Начисленные процентные расходы по срочным вкладам (депозитам)	Пассивный
7327	Фонд развития банка	Пассивный

Рисунок 6.2 – Справочник «План счетов»

Форма заключения договора приведена на рисунке 6.3. Клиент выбирается из списка, полученного от модуля «Клиенты», вид депозита и валюта — из справочников; ставка, дата начала и дата окончания подставляются автоматически. На рисунке 6.4 показан контроль данных: номер договора не соответствует формату, сумма меньше минимальной, срок больше допустимого по программе.

17

Банк «Решение» учебная АБС Клиенты Счета **Депозиты** Кредиты Банкомат

Депозитные договоры
Вклады физических лиц: заключение договора, начисление и выплата процентов

БАНКОВСКИЙ ДЕНЬ 01.10.2026 **Заккрыть день** 30 **Заккрыть дней**

Новый депозитный договор

Клиент * Лебедев Артем Владимирович (паспорт МР 3141592) Номер договора * Д-000001

Валюта * BYN — Белорусский рубль Вид депозита * «Свободное накопление» — отзывный, до востребования, 7 % годовых
Отзывный вклад до востребования, проценты выплачиваются ежемесячно. Сумма от 100,00 BYN, срок 1–12 мес.

Сумма вклада, BYN * 10000 Срок договора, мес. * 3 Процент по вкладу, % годовых 7

Дата начала 01.10.2026 Дата окончания 01.01.2027

Заклучить договор Отмена

Рисунок 6.3 – Форма заключения депозитного договора

Банк «Решение» учебная АБС Клиенты Счета **Депозиты** Кредиты Банкомат

Депозитные договоры
Вклады физических лиц: заключение договора, начисление и выплата процентов

БАНКОВСКИЙ ДЕНЬ 01.10.2026 **Заккрыть день** 30 **Заккрыть дней**

Новый депозитный договор

Форма заполнена с ошибками — исправьте отмеченные поля

Клиент * Лебедев Артем Владимирович (паспорт МР 3141592) Номер договора * Д-12
Формат номера договора: Д-000001

Валюта * BYN — Белорусский рубль Вид депозита * «Online-решение» — срочный безотзывный, 12.9 % годовых
Срочный безотзывный вклад, проценты выплачиваются в конце срока. Сумма от 500,00 BYN, срок 3–13 мес.

Сумма вклада, BYN * 100 Срок договора, мес. * 24 Процент по вкладу, % годовых 12.9
Минимальная сумма вклада — 500,00 BYN Срок по программе: от 3 до 13 мес.

Дата начала 01.10.2026 Дата окончания 01.10.2028

Заклучить договор Отмена

Рисунок 6.4 – Контроль корректности данных договора

После заключения договора открывается его карточка (рисунок 6.5): созданы счета 3404 и 3470 на имя клиента, проведены три операции приёма вклада. У счёта вклада обороты по дебету и кредиту равны 10 000, сальдо нулевое — деньги переданы в фонд развития, как и предписывает схема.

Банк «Решение» учебная АБС

Клиенты

Счета

Депозиты

Кредиты

Банкомат

Депозитные договоры

Вклады физических лиц: заключение договора, начисление и выплата процентов

БАНКОВСКИЙ ДЕНЬ 01.10.2026

Заккрыть день

30

Заккрыть дней

Договор Д-000001 заключён, открыты счета 3404000010017 и 3470000010025

Договор Д-000001 · Лебедевич Артем Владимирович

Отозвать вклад

К списку

ВИД ДЕПОЗИТА

«Свободное накопление»

Отзывный, до востребования

СУММА ВКЛАДА

10 000,00 BYN

СТАВКА

7 % годовых

СТАТУС

Действует

СРОК

3 мес.

ПЕРИОД

01.10.2026 — 01.01.2027

НАЧИСЛЕНО ПРОЦЕНТОВ

0,00 BYN

ВЫПЛАЧЕНО ПРОЦЕНТОВ

0,00 BYN

Счета договора

НОМЕР СЧЁТА	КОД	НАЗНАЧЕНИЕ	АКТИВНОСТЬ	ДЕБЕТ	КРЕДИТ	САЛЬДО
3404000010017	3404	Вклады (депозиты) до востребования физических лиц Лебедевич Артем Владимирович	Пассивный	10 000,00	10 000,00	0,00
3470000010025	3470	Начисленные процентные расходы по вкладам (депозитам) до востребования Лебедевич Артем Владимирович	Пассивный	0,00	0,00	0,00

Проводки по договору

ДАТА	ОПЕРАЦИЯ	СЧЁТ	ДЕБЕТ	КРЕДИТ
01.10.2026	Использование денег банком	3404000010017 3404	10 000,00	
		7327000000029 7327		10 000,00
01.10.2026	Перевод денег с кассы на текущий счёт	1010000000019 1010		10 000,00
		3404000010017 3404		10 000,00
01.10.2026	Внесение денег в кассу	1010000000019 1010	10 000,00	

Рисунок 6.5 – Карточка договора: счета и проводки приёма вклада

Вторым договором открыт срочный вклад «Online-решение» на 5 000 BYN. В отчёте по счетам (рисунок 6.6) теперь шесть счетов: касса, фонд развития и по два счёта на каждый договор. Фонд развития вырос на сумму обоих вкладов до 1 015 000 BYN.

Банк «Решение» учебная АБС

Клиенты

Счета

Депозиты

Кредиты

Банкомат

Счета банка

Отчёт о состоянии счетов и процедура «Закрытие банковского дня»

БАНКОВСКИЙ ДЕНЬ 01.10.2026

Закрыть день

30

Закрыть дней

ОТЧЁТ НА ДАТУ

01.10.2026

СЧЕТОВ В ОТЧЁТЕ

6

ФОНД РАЗВИТИЯ, BYN

1 015 000,00

КАССА, BYN

0,00

Отчёт по счетам

Журнал проводок

План счетов

BYN

USD

EUR

НОМЕР СЧЁТА	КОД	НАЗВАНИЕ СЧЁТА	АКТИВНОСТЬ	ДОГОВОР	ДЕБЕТ	КРЕДИТ	САЛДО
1010000000019	1010	Касса банка, BYN Денежные средства в кассе	Активный		15 000,00	15 000,00	0,00
3404000010017	3404	Лебедев Артем Владимирович Вклады (депозиты) до востребования физических лиц	Пассивный	Д-000001	10 000,00	10 000,00	0,00
3414000020015	3414	Савицкая Марина Олеговна Срочные вклады (депозиты) физических лиц	Пассивный	Д-000002	5 000,00	5 000,00	0,00
3470000010025	3470	Лебедев Артем Владимирович Начисленные процентные расходы по вкладам (депозитам) до востребования	Пассивный	Д-000001	0,00	0,00	0,00
3471000020025	3471	Савицкая Марина Олеговна Начисленные процентные расходы по срочным вкладам (депозитам)	Пассивный	Д-000002	0,00	0,00	0,00
7327000000029	7327	Фонд развития банка, BYN Фонд развития банка	Пассивный		0,00	1 015 000,00	1 015 000,00
Итого обороты, BYN					30 000,00	1 045 000,00	

Рисунок 6.6 – Отчёт по счетам при двух активных депозитных программах

Процедура «Закрытие банковского дня» выполнена 31 раз подряд (рисунок 6.7). По отзывному вкладу проценты за месяц в размере 59,45 BYN начислены и выплачены, процентный счёт обнулился. По срочному вкладу начислено $5000 \cdot 12,9 \cdot 31 / 36500 = 54,78$ BYN, и эта сумма остаётся на счёте 3471 до конца срока. Фонд развития уменьшился на сумму начисленных процентов.

20

Банк «Решение» учебная АБС

Клиенты

Счета

Депозиты

Кредиты

Банкомат

Счета банка

Отчёт о состоянии счетов и процедура «Закрытие банковского дня»

БАНКОВСКИЙ ДЕНЬ 01.11.2026

Закрыть день

31

Закрыть дней

ОТЧЁТ НА ДАТУ

01.11.2026

СЧЕТОВ В ОТЧЁТЕ

6

ФОНД РАЗВИТИЯ, BYN

1 014 885,77

КАССА, BYN

0,00

Протокол закрытия дня

27.10.2026: Д-000002 — начислены проценты 1.77 BYN
28.10.2026: Д-000001 — начислены проценты 1.92 BYN
28.10.2026: Д-000002 — начислены проценты 1.76 BYN
29.10.2026: Д-000001 — начислены проценты 1.92 BYN
29.10.2026: Д-000002 — начислены проценты 1.77 BYN
30.10.2026: Д-000001 — начислены проценты 1.92 BYN
30.10.2026: Д-000002 — начислены проценты 1.77 BYN
31.10.2026: Д-000001 — начислены проценты 1.91 BYN
31.10.2026: Д-000002 — начислены проценты 1.76 BYN
01.11.2026: Д-000001 — начислены проценты 1.92 BYN
01.11.2026: Д-000001 — выплачены проценты 59.45 BYN
01.11.2026: Д-000002 — начислены проценты 1.77 BYN
... всего записей: 63

Отчёт по счетам

Журнал проводок

План счетов

BYN

USD

EUR

НОМЕР СЧЁТА	КОД	НАЗВАНИЕ СЧЁТА	АКТИВНОСТЬ	ДОГОВОР	ДЕБЕТ	КРЕДИТ	САЛЬДО
1010000000019	1010	Касса банка, BYN Денежные средства в кассе	Активный		15 059,45	15 059,45	0,00
3404000010017	3404	Лебедевич Артем Владимирович Вклады (депозиты) до востребования физических лиц	Пассивный	Д-000001	10 000,00	10 000,00	0,00
3414000020015	3414	Савицкая Марина Олеговна Срочные вклады (депозиты) физических лиц	Пассивный	Д-000002	5 000,00	5 000,00	0,00
3470000010025	3470	Лебедевич Артем Владимирович Начисленные процентные расходы по вкладам (депозитам) до востребования	Пассивный	Д-000001	59,45	59,45	0,00
3471000020025	3471	Савицкая Марина Олеговна Начисленные процентные расходы по срочным вкладам (депозитам)	Пассивный	Д-000002	0,00	54,78	54,78
7327000000029	7327	Фонд развития банка, BYN Фонд развития банка	Пассивный		114,23	1 015 000,00	1 014 885,77
Итого обороты, BYN					30 233,13	1 045 173,68	

Рисунок 6.7 – Отчёт по счетам через месяц и протокол закрытия дня

Карточка отзывного вклада через месяц и журнал проводок главной книги показаны на рисунках 6.8 и 6.9: видны ежедневные начисления, перевод процентов в кассу и их выдача.

21

Банк «Решение» учебная АБС

Клиенты

Счета

Депозиты

Кредиты

Банкомат

Депозитные договоры

Вклады физических лиц: заключение договора, начисление и выплата процентов

БАНКОВСКИЙ ДЕНЬ 01.11.2026

Закрыть день

30

Закрыть дней

Договор Д-000001 · Лебедев Артем Владимирович

Отозвать вклад

К списку

ВИД ДЕПОЗИТА

«Свободное накопление»

Отзывный, до востребования

СУММА ВКЛАДА

10 000,00 BYN

СТАВКА

7 % годовых

СТАТУС

Действует

СРОК

3 мес.

ПЕРИОД

01.10.2026 — 01.01.2027

НАЧИСЛЕНО ПРОЦЕНТОВ

59,45 BYN

ВЫПЛАЧЕНО ПРОЦЕНТОВ

59,45 BYN

Счета договора

НОМЕР СЧЁТА	КОД	НАЗНАЧЕНИЕ	АКТИВНОСТЬ	ДЕБЕТ	КРЕДИТ	САЛДО
3484000010017	3404	Вклады (депозиты) до востребования физических лиц Лебедев Артем Владимирович	Пассивный	10 000,00	10 000,00	0,00
3470000010025	3470	Начисленные процентные расходы по вкладам (депозитам) до востребования Лебедев Артем Владимирович	Пассивный	59,45	59,45	0,00

Проводки по договору

ДАТА	ОПЕРАЦИЯ	СЧЁТ	ДЕБЕТ	КРЕДИТ
01.11.2026	Вывод процентов из кассы	1010000000019 1010		59,45
01.11.2026	Перевод процентов в кассу	3470000010025 3470	59,45	
		1010000000019 1010	59,45	
01.11.2026	Начисление процентов по депозиту	7327000000029 7327	1,92	
		3470000010025 3470		1,92
31.10.2026	Начисление процентов по депозиту	7327000000029 7327	1,91	
		3470000010025 3470		1,91
30.10.2026	Начисление процентов по депозиту	7327000000029 7327	1,92	
		3470000010025 3470		1,92
29.10.2026	Начисление процентов по депозиту	7327000000029 7327	1,92	
		3470000010025 3470		1,92
28.10.2026	Начисление процентов по депозиту	7327000000029 7327	1,92	
		3470000010025 3470		1,92
27.10.2026	Начисление процентов по депозиту	7327000000029 7327	1,91	

Рисунок 6.8 – Карточка отзывного вклада после выплаты процентов

Банк «Решение» учебная АБС

Клиенты

Счета

Депозиты

Кредиты

Банкомат

Счета банка

Отчёт о состоянии счетов и процедура «Закрытие банковского дня»

БАНКОВСКИЙ ДЕНЬ 01.11.2026

Закрыть день

31

Закрыть дней

ОТЧЁТ НА ДАТУ

01.11.2026

СЧЕТОВ В ОТЧЁТЕ

6

ФОНД РАЗВИТИЯ, BYN

1 014 885,77

КАССА, BYN

0,00

Протокол закрытия дня

27.10.2026: Д-000002 — начислены проценты 1.77 BYN
28.10.2026: Д-000001 — начислены проценты 1.92 BYN
28.10.2026: Д-000002 — начислены проценты 1.76 BYN
29.10.2026: Д-000001 — начислены проценты 1.92 BYN
29.10.2026: Д-000002 — начислены проценты 1.77 BYN
30.10.2026: Д-000001 — начислены проценты 1.92 BYN
30.10.2026: Д-000002 — начислены проценты 1.77 BYN
31.10.2026: Д-000001 — начислены проценты 1.91 BYN
31.10.2026: Д-000002 — начислены проценты 1.76 BYN
01.11.2026: Д-000001 — начислены проценты 1.92 BYN
01.11.2026: Д-000001 — выплачены проценты 59.45 BYN
01.11.2026: Д-000002 — начислены проценты 1.77 BYN
... всего записей: 63

Отчёт по счетам

Журнал проводок

План счетов

№	ДАТА	ОПЕРАЦИЯ	ДОГОВОР	СЧЁТ	ДЕБЕТ	КРЕДИТ
73	01.11.2026	Начисление процентов по депозиту	Д-000002	73270000000029 Фонд развития банка, BYN	1,77	
				3471000020025 Савицкая Марина Олеговна		1,77
72	01.11.2026	Вывод процентов из кассы	Д-000001	10100000000019 Касса банка, BYN		59,45
71	01.11.2026	Перевод процентов в кассу	Д-000001	3470000010025 Лебедев Артем Владимирович	59,45	
				10100000000019 Касса банка, BYN	59,45	
70	01.11.2026	Начисление процентов по депозиту	Д-000001	73270000000029 Фонд развития банка, BYN	1,92	
				3470000010025 Лебедев Артем Владимирович		1,92
69	31.10.2026	Начисление процентов по депозиту	Д-000002	73270000000029 Фонд развития банка, BYN	1,76	
				3471000020025 Савицкая Марина Олеговна		1,76
68	31.10.2026	Начисление процентов по депозиту	Д-000001	73270000000029 Фонд развития банка, BYN	1,91	

Рисунок 6.9 – Журнал проводок главной книги

После закрытия ещё 61 дня наступает 1 января 2027 года — дата окончания обоих договоров. Вклады возвращены, договоры закрыты (рисунок 6.10). По отзывному вкладу за 92 дня выплачено 176,44 BYN, по срочному — 162,58 BYN одной суммой. В итоговом отчёте (рисунок 6.11) сальдо всех клиентских счетов и кассы нулевое, а фонд развития равен стартовому капиталу за вычетом выплаченных процентов: $1\,000\,000 - 176,44 - 162,58 = 999\,660,98$ BYN.

23

Банк «Решение»

учебная АБС

Клиенты

Счета

Депозиты

Кредиты

Банкомат

Депозитные договоры

Вклады физических лиц: заключение договора, начисление и выплата процентов

БАНКОВСКИЙ ДЕНЬ 01.01.2027

Заккрыть день

61

Заккрыть дней

Протокол закрытия дня

31.12.2026: Д-000001 — начислены проценты 1.92 BYN

31.12.2026: Д-000002 — начислены проценты 1.77 BYN

01.01.2027: Д-000001 — начислены проценты 1.92 BYN

01.01.2027: Д-000001 — выплачены проценты 59.45 BYN

01.01.2027: Д-000001 — срок договора истёк, вклад 10000.00 BYN возвращён

01.01.2027: Д-000002 — начислены проценты 1.77 BYN

01.01.2027: Д-000002 — выплачены проценты 162.58 BYN

01.01.2027: Д-000002 — срок договора истёк, вклад 5000.00 BYN возвращён

... всего записей: 127

Договоров: 2

Заклучить договор

ДОГОВОР	КЛИЕНТ	ВИД ДЕПОЗИТА	СУММА	СТАВКА	НАЧАЛО	ОКОНЧАНИЕ	НАЧИСЛЕНО	ВЫПЛАЧЕНО	СТАТУС
Д-000002	Савицкая Марина Олеговна	Online-решение Срочный безотзывный	5 000,00 BYN	12.9 %	01.10.2026	01.01.2027	162,58	162,58	Закрыт
Д-000001	Лебедев Артем Владимирович	Свободное накопление Отзывный, до востребования	10 000,00 BYN	7 %	01.10.2026	01.01.2027	176,44	176,44	Закрыт

Рисунок 6.10 – Список договоров после окончания срока

Банк «Решение»

учебная АБС

Клиенты

Счета

Депозиты

Кредиты

Банкомат

Счета банка

Отчёт о состоянии счетов и процедура «Закрытие банковского дня»

БАНКОВСКИЙ ДЕНЬ 01.01.2027

Заккрыть день

30

Заккрыть дней

ОТЧЁТ НА ДАТУ

01.01.2027

СЧЕТОВ В ОТЧЁТЕ

6

ФОНД РАЗВИТИЯ, BYN

999 660,98

КАССА, BYN

0,00

Отчёт по счетам

Журнал проводок

План счетов

BYN

USD

EUR

НОМЕР СЧЁТА	КОД	НАЗВАНИЕ СЧЁТА	АКТИВНОСТЬ	ДОГОВОР	ДЕБЕТ	КРЕДИТ	САЛДО
1010000000019	1010	Касса банка, BYN Денежные средства в кассе	Активный		30 339,02	30 339,02	0,00
3404000010017	3404	Лебедев Артем Владимирович Вклады (депозиты) до востребования физических лиц	Пассивный	Д-000001	20 000,00	20 000,00	0,00
3414000020015	3414	Савицкая Марина Олеговна Срочные вклады (депозиты) физических лиц	Пассивный	Д-000002	10 000,00	10 000,00	0,00
3470000010025	3470	Лебедев Артем Владимирович Начисленные процентные расходы по вкладам (депозитам) до востребования	Пассивный	Д-000001	176,44	176,44	0,00
3471000020025	3471	Савицкая Марина Олеговна Начисленные процентные расходы по срочным вкладам (депозитам)	Пассивный	Д-000002	162,58	162,58	0,00
7327000000029	7327	Фонд развития банка, BYN Фонд развития банка	Пассивный		15 339,02	1 015 000,00	999 660,98
Итого обороты, BYN					76 017,06	1 075 678,04	

Рисунок 6.11 – Итоговый отчёт по счетам

24

ЗАКЛЮЧЕНИЕ

В работе реализованы два микросервиса: главная книга банка и сервис депозитных операций. Главная книга ведёт справочник «План счетов», открывает лицевые счета с 13-значным номером и контрольным ключом, проводит операции атомарными пакетами и формирует отчёт о состоянии счетов. Сервис депозитов поддерживает две программы банка «Решение» — отзывный вклад с ежемесячной выплатой процентов и срочный безотзывный вклад с выплатой в конце срока; при заключении договора он открывает два счёта клиента и выполняет проводки строго по схеме из задания.

Процедура «Закрытие банковского дня» построена как взаимодействие сервисов: главная книга объявляет новую дату, сервис депозитов начисляет и выплачивает проценты. Идемпотентные ключи проводок, сдвиг даты после успешной обработки и накопительный расчёт процентов делают процедуру устойчивой к повторам и к временной недоступности участника — это те свойства, без которых распределённая система не может надёжно работать с деньгами.

Данные о клиентах берутся из модуля первой работы по REST. Корректность проверена 69 автоматическими тестами, в том числе сверкой оборотов всех счетов со схемой проводок на полном жизненном цикле договора.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Об установлении Плана счетов бухгалтерского учёта в банках и небанковских кредитно-финансовых организациях Республики Беларусь: постановление Правления Национального банка Республики Беларусь от 29.08.2013 № 506 [Электронный ресурс]. — Режим доступа: <https://www.nbrb.by/legislation/accountingreporting>. — Дата доступа: 01.10.2026.
2. Вклады для физических лиц / ЗАО «Банк «Решение» [Электронный ресурс]. — Режим доступа: <https://rbank.by/life/deposits/>. — Дата доступа: 01.10.2026.
3. Ньюмен, С. Создание микросервисов / С. Ньюмен. — 2-е изд. — СПб.: Питер, 2023. — 624 с.
4. Ричардсон, К. Микросервисы. Паттерны разработки и рефакторинга / К. Ричардсон. — СПб.: Питер, 2019. — 544 с.
5. Spring Framework Reference: REST Clients, HTTP Interface [Электронный ресурс]. — Режим доступа: <https://docs.spring.io/spring-framework/reference/integration/rest-clients.html>. — Дата доступа: 01.10.2026.

ПРИЛОЖЕНИЕ А (обязательное)

Исходный код сервисов счетов и депозитов

Общая библиотека сервисов (bank-common)

Листинг А.1 – bank-common/pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>by.bsuir.bank</groupId>
    <artifactId>bank</artifactId>
    <version>1.0.0</version>
  </parent>

  <artifactId>bank-common</artifactId>
  <name>bank-common</name>
  <description>Общая библиотека сервисов: формат ошибок и REST-клиенты смежных микросервисов</description>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
      <optional>true</optional>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <!-- test-jar: тестовый двойник главной книги InMemoryLedger для тестов сервисов депозитов и кредитов -->
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-jar-plugin</artifactId>
        <executions>
          <execution>
            <goals>
              <goal>test-jar</goal>
            </goals>
          </execution>
        </executions>
      </plugin>
    </plugins>
  </build>
</project>
```

Листинг А.2 –

bank-common/src/main/java/by/bsuir/bank/common/HttpApis.java

```
package by.bsuir.bank.common;

import org.springframework.web.client.RestClient;
import org.springframework.web.client.support.RestClientAdapter;
import org.springframework.web.service.invoker.HttpServiceProxyFactory;

/** Создание REST-клиента смежного сервиса по интерфейсу с аннотациями @HttpExchange. */
public final class HttpApis {

    public static <T> T proxy(RestClient.Builder builder, String baseUrl, Class<T> api) {
```

```

        RestClient client = builder.baseUrl(baseUrl).build();
        return HttpServiceProxyFactory.builderFor(RestClientAdapter.create(client)).build().createClient(api);
    }

    private HttpApis() {
    }
}

```

Листинг А.3 –

bank-common/src/main/java/by/bsuir/bank/common/IntegrationConfig.java

```

package by.bsuir.bank.common;

import by.bsuir.bank.common.client.ClientApi;
import by.bsuir.bank.common.ledger.LedgerApi;
import org.springframework.boot.context.properties.EnableConfigurationProperties;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.client.RestClient;

/** REST-клиенты сервисов «Клиенты» и «Счета» для сервисов депозитов и кредитов. */
@Configuration
@EnableConfigurationProperties(ServiceUrls.class)
public class IntegrationConfig {

    @Bean
    public ClientApi clientApi(RestClient.Builder builder, ServiceUrls urls) {
        return HttpApis.proxy(builder, urls.clients(), ClientApi.class);
    }

    @Bean
    public LedgerApi ledgerApi(RestClient.Builder builder, ServiceUrls urls) {
        return HttpApis.proxy(builder, urls.accounts(), LedgerApi.class);
    }
}

```

Листинг А.4 –

bank-common/src/main/java/by/bsuir/bank/common/ServiceUrls.java

```

package by.bsuir.bank.common;

import org.springframework.boot.context.properties.ConfigurationProperties;

/** Адреса смежных микросервисов (bank.services.* в application.yml, переопределяются переменными окружения). */
@ConfigurationProperties("bank.services")
public record ServiceUrls(String clients, String accounts, String deposits, String credits) {
}

```

Листинг А.5 –

bank-common/src/main/java/by/bsuir/bank/common/api/ApiError.java

```

package by.bsuir.bank.common.api;

import java.util.Map;

/** Единый формат ошибки всех сервисов: машинный код, сообщение и сообщения по полям запроса. */
public record ApiError(String code, String message, Map<String, String> fields) {
}

```

Листинг А.6 –

bank-common/src/main/java/by/bsuir/bank/common/api/BankException.java

```

package by.bsuir.bank.common.api;

import lombok.Getter;
import org.springframework.http.HttpStatus;

import java.util.Map;

/** Нарушение бизнес-правила: превращается обработчиком в ответ с кодом состояния и телом ApiError. */
@Getter
public class BankException extends RuntimeException {
}

```

```

private final HttpStatus status;
private final String code;
private final Map<String, String> fields;

public BankException(HttpStatus status, String code, String message, Map<String, String> fields) {
    super(message);
    this.status = status;
    this.code = code;
    this.fields = fields;
}

public BankException(HttpStatus status, String code, String message) {
    this(status, code, message, Map.of());
}

public static BankException notFound(String message) {
    return new BankException(HttpStatus.NOT_FOUND, "NOT_FOUND", message);
}

public static BankException invalid(String message) {
    return new BankException(HttpStatus.BAD_REQUEST, "INVALID_REQUEST", message);
}

public static BankException conflict(String code, String message) {
    return new BankException(HttpStatus.CONFLICT, code, message);
}

/** Ошибки, привязанные к полям формы. */
public static BankException fields(Map<String, String> fields) {
    return new BankException(HttpStatus.BAD_REQUEST, "INVALID_REQUEST",
        "Проверьте правильность заполнения полей", fields);
}
}

```

Листинг А.7 – bank-

common/src/main/java/by/bsuir/bank/common/api/CommonExceptionHandler.java

```

package by.bsuir.bank.common.api;

import org.springframework.http.HttpStatus;
import org.springframework.http.HttpStatusCode;
import org.springframework.http.ResponseEntity;
import org.springframework.http.converter.HttpMessageNotReadableException;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.client.ResourceAccessException;
import org.springframework.web.client.RestClientResponseException;

import java.util.LinkedHashMap;
import java.util.Map;

/** Общий обработчик ошибок REST-контроллеров. */
@RestControllerAdvice
public class CommonExceptionHandler {

    public static final String REQUIRED = "Обязательное поле";

    @ExceptionHandler(BankException.class)
    public ResponseEntity<ApiError> business(BankException e) {
        return ResponseEntity.status(e.getStatus()).body(new ApiError(e.getCode(), e.getMessage(), e.getFields()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ApiError invalid(MethodArgumentNotValidException e) {
        Map<String, String> fields = new LinkedHashMap<>();
        for (FieldError error : e.getBindingResult().getFieldErrors()) {
            fields.merge(error.getField(), String.valueOf(error.getDefaultMessage()),
                (old, fresh) -> REQUIRED.equals(fresh) ? fresh : old);
        }
        return new ApiError("INVALID_REQUEST", "Проверьте правильность заполнения полей", fields);
    }

    @ExceptionHandler(HttpMessageNotReadableException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ApiError unreadable(HttpMessageNotReadableException e) {
        return new ApiError("INVALID_REQUEST", "Некорректный формат запроса", Map.of());
    }
}

```

```

    }

    /** Смежный сервис ответил ошибкой: его сообщение передаётся дальше, 5xx превращается в 502. */
    @ExceptionHandler(RestClientResponseException.class)
    public ResponseEntity<ApiError> downstream(RestClientResponseException e) {
        ApiError body = null;
        try {
            body = e.getResponseBodyAs(ApiError.class);
        } catch (RuntimeException ignored) {
            // тело ответа не в формате ApiError
        }
        if (body == null || body.message() == null) {
            body = new ApiError("DOWNSTREAM_ERROR", "Ошибка смежного сервиса", Map.of());
        }
        HttpStatusCode status = e.getStatusCode().is4xxClientError() ? e.getStatusCode() : HttpStatus.BAD_GATEWAY;
        return ResponseEntity.status(status).body(new ApiError(body.code(), body.message(), Map.of()));
    }

    /** Смежный сервис не отвечает. */
    @ExceptionHandler(ResourceAccessException.class)
    @ResponseStatus(HttpStatus.SERVICE_UNAVAILABLE)
    public ApiError unavailable(ResourceAccessException e) {
        return new ApiError("SERVICE_UNAVAILABLE", "Смежный сервис недоступен, повторите операцию позже", Map.of());
    }
}

```

Листинг А.8 –

bank-common/src/main/java/by/bsuir/bank/common/client/ClientApi.java

```

package by.bsuir.bank.common.client;

import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.service.annotation.GetExchange;
import org.springframework.web.service.annotation.HttpExchange;

import java.util.List;

/** REST-клиент модуля «Клиенты» (ЛР1). */
@HttpExchange("/api/clients")
public interface ClientApi {

    @GetExchange
    List<ClientInfo> list();

    @GetExchange("/{id}")
    ClientInfo get(@PathVariable Long id);
}

```

Листинг А.9 –

bank-common/src/main/java/by/bsuir/bank/common/client/ClientInfo.java

```

package by.bsuir.bank.common.client;

/** Сведения о клиенте, которые нужны договорам: остальные поля ответа сервиса «Клиенты» игнорируются. */
public record ClientInfo(Long id, String fullName, String passportSeries, String passportNumber,
                        String identificationNumber) {
}

```

Листинг А.10 –

bank-common/src/main/java/by/bsuir/bank/common/ledger/AccountInfo.java

```

package by.bsuir.bank.common.ledger;

import java.math.BigDecimal;

public record AccountInfo(String number, String chartCode, String chartName, String activity, String activityTitle,
                        String currency, String name, Integer clientId, String contractRef,
                        BigDecimal debit, BigDecimal credit, BigDecimal balance) {
}

```

Листинг А.11 –

bank-common/src/main/java/by/bsuir/bank/common/ledger/BankDayInfo.java

```
package by.bsuir.bank.common.ledger;

import java.time.LocalDate;

public record BankDayInfo(LocalDate date) {
}
```

Листинг А.12 – bank-common/src/main/java/by/bsuir/bank/common/ledger/Batch.java

```
package by.bsuir.bank.common.ledger;

import java.time.LocalDate;
import java.util.List;

/** Пакет операций, проводимый главной книгой атомарно; batchKey – ключ идемпотентности. */
public record Batch(String batchKey, String contractRef, LocalDate bankDate, List<Posting> operations) {

    public static Batch of(String batchKey, String contractRef, LocalDate bankDate, Posting... operations) {
        return new Batch(batchKey, contractRef, bankDate, List.of(operations));
    }
}
```

Листинг А.13 – bank-common/src/main/java/by/bsuir/bank/common/ledger/CurrencyInfo.java

```
package by.bsuir.bank.common.ledger;

public record CurrencyInfo(String code, String name) {
}
```

Листинг А.14 – bank-common/src/main/java/by/bsuir/bank/common/ledger/DayCloseInfo.java

```
package by.bsuir.bank.common.ledger;

import java.time.LocalDate;
import java.util.List;

public record DayCloseInfo(LocalDate bankDate, List<String> events) {
}
```

Листинг А.15 – bank-common/src/main/java/by/bsuir/bank/common/ledger/Entry.java

```
package by.bsuir.bank.common.ledger;

import java.math.BigDecimal;

/** Строка проводки: счёт, сторона и сумма. */
public record Entry(String account, Side side, BigDecimal amount) {

    public static Entry debit(String account, BigDecimal amount) {
        return new Entry(account, Side.DEBIT, amount);
    }

    public static Entry credit(String account, BigDecimal amount) {
        return new Entry(account, Side.CREDIT, amount);
    }
}
```

Листинг А.16 – bank-common/src/main/java/by/bsuir/bank/common/ledger/LedgerApi.java

```
package by.bsuir.bank.common.ledger;

import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.service.annotation.GetExchange;
import org.springframework.web.service.annotation.HttpExchange;
```

```

import org.springframework.web.service.annotation.PostExchange;

import java.util.List;

/** REST-клиент сервиса «Счета»: банковский день, счета и проводки. */
@HttpExchange("/api")
public interface LedgerApi {

    String CASH = "1010";
    String FUND = "7327";

    @GetExchange("/bank-day")
    BankDayInfo bankDay();

    @PostExchange("/bank-day/close")
    DayCloseInfo closeDay(@RequestParam int days);

    @GetExchange("/currencies")
    List<CurrencyInfo> currencies();

    /** Собственный счёт банка: касса (1010) или фонд развития (7327) в заданной валюте. */
    @GetExchange("/accounts/system")
    AccountInfo systemAccount(@RequestParam String chartCode, @RequestParam String currency);

    @GetExchange("/accounts")
    List<AccountInfo> accounts(@RequestParam String contractRef);

    @GetExchange("/accounts/{number}")
    AccountInfo account(@PathVariable String number);

    @PostExchange("/accounts")
    AccountInfo open(@RequestBody OpenAccount request);

    @PostExchange("/operations")
    List<OperationInfo> post(@RequestBody Batch batch);

    @GetExchange("/operations")
    List<OperationInfo> operations(@RequestParam String contractRef);
}

```

Листинг А.17 –

bank-common/src/main/java/by/bsuir/bank/common/ledger/OpenAccount.java

```

package by.bsuir.bank.common.ledger;

public record OpenAccount(String chartCode, String currency, Integer clientId, String name, String contractRef) {
}

```

Листинг А.18 –

bank-common/src/main/java/by/bsuir/bank/common/ledger/OperationInfo.java

```

package by.bsuir.bank.common.ledger;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;

public record OperationInfo(Long id, LocalDate bankDate, String description, String contractRef,
    List<EntryInfo> entries) {

    public record EntryInfo(String account, String accountName, String chartCode, Side side, BigDecimal amount) {
    }
}

```

Листинг А.19 –

bank-common/src/main/java/by/bsuir/bank/common/ledger/Posting.java

```

package by.bsuir.bank.common.ledger;

import java.util.List;

/** Одна операция – строка схемы бухгалтерских проводок. */
public record Posting(String description, List<Entry> entries) {

    public static Posting of(String description, Entry... entries) {
    }
}

```

```

        return new Posting(description, List.of(entries));
    }
}

```

Листинг A.20 –

bank-common/src/main/java/by/bsuir/bank/common/ledger/Side.java

```

package by.bsuir.bank.common.ledger;

public enum Side {
    DEBIT, CREDIT
}

```

Листинг A.21 –

bank-common/src/test/java/by/bsuir/bank/common/ledger/InMemoryLedger.java

```

package by.bsuir.bank.common.ledger;

import org.springframework.http.HttpStatus;
import org.springframework.web.client.HttpClientErrorException;

import java.math.BigDecimal;
import java.nio.charset.StandardCharsets;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/**
 * Тестовый двойник сервиса «Счета»: главная книга в памяти с теми же правилами
 * (сальдо по активности, запрет отрицательного остатка, атомарность и идемпотентность пакета).
 * Позволяет тестировать сервисы депозитов и кредитов без запуска смежного микросервиса.
 */
public class InMemoryLedger implements LedgerApi {

    public static final LocalDate START = LocalDate.of(2026, 10, 1);
    public static final BigDecimal CAPITAL = new BigDecimal("1000000.00");

    private final Map<String, AccountInfo> accounts = new LinkedHashMap<>();
    private final Map<String, List<OperationInfo>> batches = new LinkedHashMap<>();
    private LocalDate date;
    private long operationSeq;

    public InMemoryLedger() {
        reset();
    }

    /** Возвращает книгу в исходное состояние: касса и фонд развития в трёх валютах, день 01.10.2026. */
    public final void reset() {
        accounts.clear();
        batches.clear();
        operationSeq = 0;
        date = START;
        for (String currency : List.of("BYN", "USD", "EUR")) {
            put(CASH, currency, null, "Касса банка, " + currency, null, BigDecimal.ZERO);
            put(FUND, currency, null, "Фонд развития банка, " + currency, null, CAPITAL);
        }
    }

    public void setDate(LocalDate date) {
        this.date = date;
    }

    public BigDecimal debit(String number) {
        return account(number).debit();
    }

    public BigDecimal credit(String number) {
        return account(number).credit();
    }

    public BigDecimal balance(String number) {
        return account(number).balance();
    }

    public List<OperationInfo> journal() {
        return batches.values().stream().flatMap(List::stream).toList();
    }
}

```

```

}

@Override
public BankDayInfo bankDay() {
    return new BankDayInfo(date);
}

@Override
public DayCloseInfo closeDay(int days) {
    date = date.plusDays(days);
    return new DayCloseInfo(date, List.of());
}

@Override
public List<CurrencyInfo> currencies() {
    return List.of(new CurrencyInfo("BYN", "Белорусский рубль"), new CurrencyInfo("EUR", "Евро"),
        new CurrencyInfo("USD", "Доллар США"));
}

@Override
public AccountInfo systemAccount(String chartCode, String currency) {
    return accounts.values().stream()
        .filter(a -> a.chartCode().equals(chartCode) && a.currency().equals(currency) && a.clientId() == null
            && a.contractRef() == null)
        .findFirst().orElseThrow();
}

@Override
public List<AccountInfo> accounts(String contractRef) {
    return accounts.values().stream().filter(a -> contractRef.equals(a.contractRef())).toList();
}

@Override
public AccountInfo account(String number) {
    AccountInfo account = accounts.get(number);
    if (account == null) {
        throw error(HttpStatus.NOT_FOUND, "NOT_FOUND", "Счёт " + number + " не найден");
    }
    return account;
}

@Override
public AccountInfo open(OpenAccount request) {
    if (request.contractRef() != null) {
        var existing = accounts.values().stream()
            .filter(a -> request.contractRef().equals(a.contractRef()) && a.chartCode().equals(request.chartCode()))
            .findFirst();
        if (existing.isPresent()) {
            return existing.get();
        }
    }
    return put(request.chartCode(), request.currency(), request.clientId(), request.name(), request.contractRef(),
        BigDecimal.ZERO);
}

@Override
public List<OperationInfo> post(Batch batch) {
    if (batches.containsKey(batch.batchKey())) {
        return batches.get(batch.batchKey());
    }
    Map<String, AccountInfo> draft = new LinkedHashMap<>(accounts);
    List<OperationInfo> posted = new ArrayList<>();
    for (Posting posting : batch.operations()) {
        List<OperationInfo.EntryInfo> entries = new ArrayList<>();
        for (Entry entry : posting.entries()) {
            AccountInfo account = draft.get(entry.account());
            if (account == null) {
                throw error(HttpStatus.NOT_FOUND, "NOT_FOUND", "Счёт " + entry.account() + " не найден");
            }
            BigDecimal debit = entry.side() == Side.DEBIT ? account.debit().add(entry.amount()) : account.debit();
            BigDecimal credit = entry.side() == Side.CREDIT ? account.credit().add(entry.amount()) : account.credit();
            draft.put(account.number(), copy(account, debit, credit));
            entries.add(new OperationInfo.EntryInfo(account.number(), account.name(), account.chartCode(),
                entry.side(), entry.amount()));
        }
        for (Entry entry : posting.entries()) {
            if (draft.get(entry.account()).balance().signum() < 0) {
                throw error(HttpStatus.CONFLICT, "INSUFFICIENT_FUNDS",
                    "Недостаточно средств на счёте " + entry.account() + " для операции «" + posting.description() + "»");
            }
        }
        posted.add(new OperationInfo(++operationSeq, batch.bankDate() != null ? batch.bankDate() : date,

```

```

        posting.description(), batch.contractRef(), entries));
    }
    accounts.putAll(draft);
    batches.put(batch.batchKey(), posted);
    return posted;
}

@Override
public List<OperationInfo> operations(String contractRef) {
    return journal().stream().filter(o -> contractRef.equals(o.contractRef())).toList();
}

private AccountInfo put(String chartCode, String currency, Integer clientId, String name, String contractRef,
    BigDecimal credit) {
    boolean passive = chartCode.startsWith("3") || chartCode.startsWith("7");
    String number = chartCode + String.format("%09d", accounts.size() + 1);
    AccountInfo account = copy(new AccountInfo(number, chartCode, "", passive ? "PASSIVE" : "ACTIVE",
        passive ? "Пассивный" : "Активный", currency, name, clientId, contractRef, null, null, null),
        BigDecimal.ZERO, credit);
    accounts.put(number, account);
    return account;
}

private static AccountInfo copy(AccountInfo a, BigDecimal debit, BigDecimal credit) {
    BigDecimal balance = "PASSIVE".equals(a.activity()) ? credit.subtract(debit) : debit.subtract(credit);
    return new AccountInfo(a.number(), a.chartCode(), a.chartName(), a.activity(), a.activityTitle(),
        a.currency(), a.name(), a.clientId(), a.contractRef(), debit, credit, balance);
}

/** Ошибка в том же виде, в каком её вернул бы настоящий сервис: код состояния и тело ApiError. */
private static HttpClientErrorException error(HttpStatus status, String code, String message) {
    String body = "{\"code\":\"" + code + "\",\"message\":\"" + message.replace("\"", "") + "\",\"fields\":{}}";
    return HttpClientErrorException.create(status, status.getReasonPhrase(), new org.springframework.http.HttpHeaders(),
        body.getBytes(StandardCharsets.UTF_8), StandardCharsets.UTF_8);
}
}

```

Микросервис «Счета» (account-service)

Листинг А.22 – account-service/pom.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>by.bsuir.bank</groupId>
        <artifactId>bank</artifactId>
        <version>1.0.0</version>
    </parent>

    <artifactId>account-service</artifactId>
    <name>account-service</name>
    <description>МР2. Микросервис «Счета»: план счетов, лицевые счета, проводки, банковский день</description>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>
        <dependency>
            <groupId>by.bsuir.bank</groupId>
            <artifactId>bank-common</artifactId>
        </dependency>
        <dependency>
            <groupId>by.bsuir.bank</groupId>
            <artifactId>ui-kit</artifactId>
        </dependency>
    </dependencies>

```

```

        </dependency>
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <optional>true</optional>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-failsafe-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>

```

Листинг A.23 – account-service/src/main/resources/application.yml

```

server:
  port: ${ACCOUNT_SERVICE_PORT:8082}

spring:
  application:
    name: account-service
  datasource:
    url: jdbc:h2:mem:accounts;DB_CLOSE_DELAY=-1
    username: sa
  sql:
    init:
      mode: always
  jpa:
    hibernate:
      ddl-auto: validate
    open-in-view: false
  h2:
    console:
      enabled: true

bank:
  fund:
    # Стартовый капитал фонда развития банка по валютам (ЛР2 – 1 млн BYN, ЛР3 – 100 млрд BYN)
    capital:
      BYN: ${BANK_FUND_CAPITAL_BYN:100000000000}
      USD: 50000000
      EUR: 50000000
  day-close:
    # Сервисы, которые обрабатывают открытие нового банковского дня: начисляют проценты, проводят платежи
    participants:
      - name: Депозиты
        url: ${DEPOSIT_SERVICE_URL:http://localhost:8083}
      - name: Кредиты
        url: ${CREDIT_SERVICE_URL:http://localhost:8084}

```

Листинг A.24 – account-service/src/main/resources/schema.sql

```

-- Схема данных главной книги банка

create table currency (
  code varchar(3) primary key,
  name varchar(60) not null
);

```

```

-- Справочник «План счетов»: балансовый счёт, наименование и активность
create table chart_of_accounts (
    code    varchar(4)    primary key,
    name    varchar(200) not null,
    activity varchar(14)  not null check (activity in ('ACTIVE', 'PASSIVE', 'ACTIVE_PASSIVE'))
);

-- Лицевые счета: 13-значный номер, обороты по дебету и кредиту
create table account (
    id          bigint generated by default as identity primary key,
    number      varchar(13) not null unique,
    chart_code  varchar(4)  not null references chart_of_accounts,
    currency_code varchar(3) not null references currency,
    name        varchar(200) not null,
    owner_code  int         not null,      -- код клиента, 0 – счёт самого банка
    owner_seq   int         not null,      -- порядковый номер счёта у владельца
    contract_ref varchar(40),             -- договор, в рамках которого открыт счёт
    debit       decimal(19, 2) not null default 0,
    credit      decimal(19, 2) not null default 0,
    opened_on   date         not null,
    constraint uq_account_owner unique (owner_code, owner_seq)
);

create index ix_account_contract on account (contract_ref);

-- Ключи идемпотентности: повтор запроса с тем же ключом не создаёт проводки второй раз
create table posting_batch (
    batch_key  varchar(80) primary key,
    created_at timestamp not null
);

-- Журнал операций и строки проводок
create table operation (
    id          bigint generated by default as identity primary key,
    batch_key   varchar(80) not null references posting_batch,
    bank_date   date         not null,
    description varchar(200) not null,
    contract_ref varchar(40),
    created_at  timestamp not null
);

create table operation_entry (
    id          bigint generated by default as identity primary key,
    operation_id bigint      not null references operation,
    account_id  bigint      not null references account,
    side        varchar(6)   not null check (side in ('DEBIT', 'CREDIT')),
    amount      decimal(19, 2) not null check (amount > 0)
);

create index ix_entry_account on operation_entry (account_id);

-- Текущий открытый банковский день (одна строка)
create table bank_day (
    id          int primary key,
    bank_date   date not null
);

```

Листинг A.25 – account-service/src/main/resources/data.sql

```

insert into currency (code, name) values
    ('BYN', 'Белорусский рубль'),
    ('USD', 'Доллар США'),
    ('EUR', 'Евро');

-- Балансовые счета, необходимые для депозитных и кредитных программ физических лиц
insert into chart_of_accounts (code, name, activity) values
    ('1010', 'Денежные средства в кассе', 'ACTIVE'),
    ('1201', 'Корреспондентский счёт в Национальном банке', 'ACTIVE'),
    ('2400', 'Кредиты физическим лицам', 'ACTIVE'),
    ('2470', 'Начисленные процентные доходы по кредитам физическим лицам', 'ACTIVE'),
    ('3012', 'Текущие (расчётные) счета коммерческих организаций', 'PASSIVE'),
    ('3014', 'Текущие (расчётные) счета физических лиц', 'PASSIVE'),
    ('3404', 'Вклады (депозиты) до востребования физических лиц', 'PASSIVE'),
    ('3414', 'Срочные вклады (депозиты) физических лиц', 'PASSIVE'),
    ('3470', 'Начисленные процентные расходы по вкладам (депозитам) до востребования', 'PASSIVE'),
    ('3471', 'Начисленные процентные расходы по срочным вкладам (депозитам)', 'PASSIVE'),
    ('7327', 'Фонд развития банка', 'PASSIVE');

insert into bank_day (id, bank_date) values (1, current_date);

```

Листинг А.26 – account-service/src/main/java/by/bsuir/bank/account/AccountServiceApplication.java

```
package by.bsuir.bank.account;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.context.properties.ConfigurationPropertiesScan;

// из общей библиотеки подключается только обработчик ошибок
@SpringBootApplication(scanBasePackages = {"by.bsuir.bank.account", "by.bsuir.bank.common.api"})
@ConfigurationPropertiesScan
public class AccountServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(AccountServiceApplication.class, args);
    }
}
```

Листинг А.27 – account-service/src/main/java/by/bsuir/bank/account/BankProperties.java

```
package by.bsuir.bank.account;

import org.springframework.boot.context.properties.ConfigurationProperties;

import java.math.BigDecimal;
import java.util.List;
import java.util.Map;

/** Настройки главной книги: стартовый капитал фонда развития и участники закрытия дня. */
@ConfigurationProperties("bank")
public record BankProperties(Fund fund, DayClose dayClose) {

    public record Fund(Map<String, BigDecimal> capital) {
    }

    public record DayClose(List<Participant> participants) {
    }

    public record Participant(String name, String url) {
    }
}
```

Листинг А.28 – account-service/src/main/java/by/bsuir/bank/account/domain/Account.java

```
package by.bsuir.bank.account.domain;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import lombok.Getter;
import lombok.Setter;

import java.math.BigDecimal;
import java.time.LocalDate;

/** Лицевой счёт: обороты по дебету и кредиту накапливаются, сальдо вычисляется по активности. */
@Entity
@Getter
@Setter
public class Account {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String number;

    @ManyToOne(fetch = FetchType.EAGER, optional = false)
    @JoinColumn(name = "chart_code")
}
```

```

private ChartAccount chart;

@Column(name = "currency_code")
private String currency;

private String name;
private int ownerCode;
private int ownerSeq;
private String contractRef;
private BigDecimal debit = BigDecimal.ZERO;
private BigDecimal credit = BigDecimal.ZERO;
private LocalDate openedOn;

public BigDecimal getBalance() {
    return chart.getActivity().balance(debit, credit);
}

public void apply(Side side, BigDecimal amount) {
    if (side == Side.DEBIT) {
        debit = debit.add(amount);
    } else {
        credit = credit.add(amount);
    }
}
}

```

Листинг А.29 – account-service/src/main/java/by/bsuir/bank/account/domain/AccountNumbers.java

```

package by.bsuir.bank.account.domain;

/**
 * Номер счёта из 13 цифр: 1–4 – балансовый счёт из плана счетов, 5–9 – код клиента,
 * 10–12 – порядковый номер счёта этого клиента, 13 – контрольный ключ.
 */
public final class AccountNumbers {

    private static final int[] WEIGHTS = {7, 1, 3};

    public static String generate(String chartCode, int ownerCode, int ownerSeq) {
        if (!chartCode.matches("\\d{4}") || ownerCode < 0 || ownerCode > 99_999 || ownerSeq < 1 || ownerSeq > 999) {
            throw new IllegalArgumentException("Номер счёта не помещается в 13 разрядов");
        }
        String body = chartCode + String.format("%05d%03d", ownerCode, ownerSeq);
        return body + checkDigit(body);
    }

    /** Контрольный ключ: взвешенная сумма первых 12 цифр (веса 7, 1, 3), умноженная на 3, по модулю 10. */
    static int checkDigit(String body) {
        int sum = 0;
        for (int i = 0; i < body.length(); i++) {
            sum += (body.charAt(i) - '0') * WEIGHTS[i % WEIGHTS.length];
        }
        return sum * 3 % 10;
    }

    public static boolean isValid(String number) {
        return number != null && number.matches("\\d{13}")
            && checkDigit(number.substring(0, 12)) == number.charAt(12) - '0';
    }

    private AccountNumbers() {
    }
}

```

Листинг А.30 – account-service/src/main/java/by/bsuir/bank/account/domain/Activity.java

```

package by.bsuir.bank.account.domain;

import java.math.BigDecimal;

/** Активность балансового счёта определяет, какая сторона увеличивает остаток. */
public enum Activity {

    ACTIVE("Активный"),
    PASSIVE("Пассивный"),
}

```

```

    ACTIVE_PASSIVE("Активно-пассивный");

    private final String title;

    Activity(String title) {
        this.title = title;
    }

    public String getTitle() {
        return title;
    }

    /** Сальдо: у активного счёта дебет минус кредит, у пассивного — кредит минус дебет. */
    public BigDecimal balance(BigDecimal debit, BigDecimal credit) {
        return this == PASSIVE ? credit.subtract(debit) : debit.subtract(credit);
    }
}

```

Листинг A.31 — account-service/src/main/java/by/bsuir/bank/account/domain/BankDay.java

```

package by.bsuir.bank.account.domain;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import lombok.Getter;
import lombok.Setter;

import java.time.LocalDate;

/** Текущий открытый банковский день — единственная строка таблицы. */
@Entity
@Getter
@Setter
public class BankDay {

    public static final int ID = 1;

    @Id
    private Integer id;

    private LocalDate bankDate;
}

```

Листинг A.32 — account-service/src/main/java/by/bsuir/bank/account/domain/ChartAccount.java

```

package by.bsuir.bank.account.domain;

import jakarta.persistence.Entity;
import jakarta.persistence.EnumType;
import jakarta.persistence.Enumerated;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import lombok.Getter;
import lombok.Setter;

/** Запись справочника «План счетов» — балансовый счёт. */
@Entity
@Table(name = "chart_of_accounts")
@Getter
@Setter
public class ChartAccount {

    @Id
    private String code;

    private String name;

    @Enumerated(EnumType.STRING)
    private Activity activity;
}

```

Листинг A.33 — account-service/src/main/java/by/bsuir/bank/account/domain/Currency.java

```

package by.bsuir.bank.account.domain;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import lombok.Getter;
import lombok.Setter;

/** Справочник валют. */
@Entity
@Getter
@Setter
public class Currency {

    @Id
    private String code;

    private String name;
}

```

Листинг А.34 – account-service/src/main/java/by/bsuir/bank/account/domain/Operation.java

```

package by.bsuir.bank.account.domain;

import jakarta.persistence.CascadeType;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.OneToMany;
import jakarta.persistence.OrderBy;
import lombok.Getter;
import lombok.Setter;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

/** Банковская операция (проводка): одна строка схемы «Депозитная/Кредитная программа». */
@Entity
@Getter
@Setter
public class Operation {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String batchKey;
    private LocalDate bankDate;
    private String description;
    private String contractRef;
    private LocalDateTime createdAt;

    @OneToMany(mappedBy = "operation", cascade = CascadeType.ALL, orphanRemoval = true)
    @OrderBy("id")
    private List<OperationEntry> entries = new ArrayList<>();

    public void addEntry(Account account, Side side, java.math.BigDecimal amount) {
        OperationEntry entry = new OperationEntry();
        entry.setOperation(this);
        entry.setAccount(account);
        entry.setSide(side);
        entry.setAmount(amount);
        entries.add(entry);
    }
}

```

Листинг А.35 – account- service/src/main/java/by/bsuir/bank/account/domain/OperationEntry.java

```

package by.bsuir.bank.account.domain;

import jakarta.persistence.Entity;
import jakarta.persistence.EnumType;
import jakarta.persistence.Enumerated;

```

```

import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.ManyToOne;
import lombok.Getter;
import lombok.Setter;

import java.math.BigDecimal;

/** Строка проводки: счёт, сторона (дебет или кредит) и сумма. */
@Entity
@Getter
@Setter
public class OperationEntry {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    private Operation operation;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    private Account account;

    @Enumerated(EnumType.STRING)
    private Side side;

    private BigDecimal amount;
}

```

Листинг А.36 – account-service/src/main/java/by/bsuir/bank/account/domain/PostingBatch.java

```

package by.bsuir.bank.account.domain;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

import java.time.LocalDateTime;

/** Ключ идемпотентности пакета проводок. */
@Entity
@Getter
@Setter
@NoArgsConstructor
public class PostingBatch {

    @Id
    private String batchKey;

    private LocalDateTime createdAt;

    public PostingBatch(String batchKey) {
        this.batchKey = batchKey;
        this.createdAt = LocalDateTime.now();
    }
}

```

Листинг А.37 – account-service/src/main/java/by/bsuir/bank/account/domain/Side.java

```

package by.bsuir.bank.account.domain;

/** Сторона счёта, по которой проходит сумма проводки. */
public enum Side {
    DEBIT, CREDIT
}

```

Листинг А.38 – account- service/src/main/java/by/bsuir/bank/account/repository/AccountRepository.java

```

package by.bsuir.bank.account.repository;

import by.bsuir.bank.account.domain.Account;
import jakarta.persistence.LockModeType;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Lock;
import org.springframework.data.jpa.repository.Query;

import java.util.List;
import java.util.Optional;

public interface AccountRepository extends JpaRepository<Account, Long> {

    Optional<Account> findByNumber(String number);

    /** Счёт блокируется на время проводки: параллельные операции не теряют обновления оборотов. */
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("select a from Account a where a.number = :number")
    Optional<Account> findByNumberForUpdate(String number);

    Optional<Account> findByContractRefAndChartCode(String contractRef, String chartCode);

    /** Собственный счёт банка (касса, фонд развития) в заданной валюте. */
    Optional<Account> findFirstByChartCodeAndCurrencyAndOwnerCodeAndContractRefIsNull(
        String chartCode, String currency, int ownerCode);

    List<Account> findAllByOrderByChartCodeAscNumberAsc();

    List<Account> findByContractRefOrderByNumber(String contractRef);

    List<Account> findByOwnerCodeOrderByNumber(int ownerCode);

    @Query("select coalesce(max(a.ownerSeq), 0) from Account a where a.ownerCode = :ownerCode")
    int maxOwnerSeq(int ownerCode);
}

```

Листинг А.39 – account- service/src/main/java/by/bsuir/bank/account/repository/BankDayRepository.java

```

package by.bsuir.bank.account.repository;

import by.bsuir.bank.account.domain.BankDay;
import org.springframework.data.jpa.repository.JpaRepository;

public interface BankDayRepository extends JpaRepository<BankDay, Integer> {
}

```

Листинг А.40 – account- service/src/main/java/by/bsuir/bank/account/repository/ChartAccountRepository.java

```

package by.bsuir.bank.account.repository;

import by.bsuir.bank.account.domain.ChartAccount;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ChartAccountRepository extends JpaRepository<ChartAccount, String> {
}

```

Листинг А.41 – account- service/src/main/java/by/bsuir/bank/account/repository/CurrencyRepository.java

```

package by.bsuir.bank.account.repository;

import by.bsuir.bank.account.domain.Currency;
import org.springframework.data.jpa.repository.JpaRepository;

public interface CurrencyRepository extends JpaRepository<Currency, String> {
}

```

Листинг А.42 – account- service/src/main/java/by/bsuir/bank/account/repository/OperationRepository.java

```

package by.bsuir.bank.account.repository;

import by.bsuir.bank.account.domain.Operation;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;

import java.util.List;

public interface OperationRepository extends JpaRepository<Operation, Long> {

    List<Operation> findByBatchKeyOrderId(String batchKey);

    @Query("""
        select distinct o from Operation o join o.entries e
        where (:contractRef is null or o.contractRef = :contractRef)
        and (:account is null or e.account.number = :account)
        order by o.id desc""")
    List<Operation> search(String contractRef, String account, Pageable page);
}

```

Листинг А.43 – account-service/src/main/java/by/bsuir/bank/account/repository/PostingBatchRepository.java

```

package by.bsuir.bank.account.repository;

import by.bsuir.bank.account.domain.PostingBatch;
import org.springframework.data.jpa.repository.JpaRepository;

public interface PostingBatchRepository extends JpaRepository<PostingBatch, String> {
}

```

Листинг А.44 – account-service/src/main/java/by/bsuir/bank/account/dto/AccountReport.java

```

package by.bsuir.bank.account.dto;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;

/** Отчёт о состоянии счетов на текущий банковский день с итогами оборотов по валютам. */
public record AccountReport(LocalDate bankDate, List<AccountView> accounts, List<Total> totals) {

    public record Total(String currency, BigDecimal debit, BigDecimal credit) {
    }
}

```

Листинг А.45 – account-service/src/main/java/by/bsuir/bank/account/dto/AccountView.java

```

package by.bsuir.bank.account.dto;

import by.bsuir.bank.account.domain.Account;
import java.math.BigDecimal;
import java.time.LocalDate;

public record AccountView(
    String number,
    String chartCode,
    String chartName,
    String activity,
    String activityTitle,
    String currency,
    String name,
    Integer clientId,
    String contractRef,
    BigDecimal debit,
    BigDecimal credit,
    BigDecimal balance,
    LocalDate openedOn
) {
}

```

```

    public static AccountView from(Account a) {
        return new AccountView(a.getNumber(), a.getChart().getCode(), a.getChart().getName(),
            a.getChart().getActivity().name(), a.getChart().getActivity().getTitle(), a.getCurrency(),
            a.getName(), a.getOwnerCode() == 0 ? null : a.getOwnerCode(), a.getContractRef(),
            a.getDebit(), a.getCredit(), a.getBalance(), a.getOpenedOn());
    }
}

```

Листинг А.46 –

account-service/src/main/java/by/bsuir/bank/account/dto/BatchRequest.java

```

package by.bsuir.bank.account.dto;

import jakarta.validation.Valid;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotEmpty;
import jakarta.validation.constraints.Size;

import java.time.LocalDate;
import java.util.List;

/**
 * Пакет операций, который проводится атомарно: либо все проводки, либо ни одной.
 * batchKey – ключ идемпотентности; bankDate – дата проводок (по умолчанию текущий банковский день).
 */
public record BatchRequest(
    @NotBlank @Size(max = 80) String batchKey,
    @Size(max = 40) String contractRef,
    LocalDate bankDate,
    @NotEmpty @Valid List<OperationRequest> operations
) {
}

```

Листинг А.47 –

account-service/src/main/java/by/bsuir/bank/account/dto/DayCloseResult.java

```

package by.bsuir.bank.account.dto;

import java.time.LocalDate;
import java.util.List;

/** Итог закрытия дня: новая дата банковского дня и протокол выполненных операций. */
public record DayCloseResult(LocalDate bankDate, List<String> events) {
}

```

Листинг А.48 –

account-service/src/main/java/by/bsuir/bank/account/dto/EntryRequest.java

```

package by.bsuir.bank.account.dto;

import by.bsuir.bank.account.domain.Side;
import jakarta.validation.constraints.Digits;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Positive;

import java.math.BigDecimal;

public record EntryRequest(
    @NotBlank String account,
    @NotNull Side side,
    @NotNull @Positive @Digits(integer = 17, fraction = 2) BigDecimal amount
) {
}

```

Листинг А.49 – account-

service/src/main/java/by/bsuir/bank/account/dto/OpenAccountRequest.java

```

package by.bsuir.bank.account.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Positive;
import jakarta.validation.constraints.Size;

```

```

/** Открытие лицевого счёта. clientId не указывается для собственных счетов банка и счетов организаций. */
public record OpenAccountRequest(
    @NotBlank String chartCode,
    @NotBlank String currency,
    @Positive Integer clientId,
    @NotBlank @Size(max = 200) String name,
    @Size(max = 40) String contractRef
) {
}

```

Листинг А.50 –

account-service/src/main/java/by/bsuir/bank/account/dto/OperationRequest.java

```

package by.bsuir.bank.account.dto;

import jakarta.validation.Valid;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotEmpty;
import jakarta.validation.constraints.Size;

import java.util.List;

public record OperationRequest(
    @NotBlank @Size(max = 200) String description,
    @NotEmpty @Valid List<EntryRequest> entries
) {
}

```

Листинг А.51 –

account-service/src/main/java/by/bsuir/bank/account/dto/OperationView.java

```

package by.bsuir.bank.account.dto;

import by.bsuir.bank.account.domain.Operation;
import by.bsuir.bank.account.domain.Side;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;

public record OperationView(
    Long id,
    LocalDate bankDate,
    String description,
    String contractRef,
    List<EntryView> entries
) {

    public record EntryView(String account, String accountName, String chartCode, Side side, BigDecimal amount) {
    }

    public static OperationView from(Operation o) {
        return new OperationView(o.getId(), o.getBankDate(), o.getDescription(), o.getContractRef(),
            o.getEntries().stream()
                .map(e -> new EntryView(e.getAccount().getNumber(), e.getAccount().getName(),
                    e.getAccount().getChart().getCode(), e.getSide(), e.getAmount()))
                .toList());
    }
}

```

Листинг А.52 – account-

service/src/main/java/by/bsuir/bank/account/service/BankDayService.java

```

package by.bsuir.bank.account.service;

import by.bsuir.bank.account.BankProperties;
import by.bsuir.bank.account.domain.BankDay;
import by.bsuir.bank.account.dto.DayCloseResult;
import by.bsuir.bank.account.repository.BankDayRepository;
import by.bsuir.bank.common.api.BankException;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Service;

```

```

import org.springframework.web.client.ResourceAccessException;
import org.springframework.web.client.RestClientException;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.locks.ReentrantLock;

/**
 * Банковский день и процедура его закрытия. Сам сервис счетов не знает правил депозитов и кредитов:
 * он объявляет сервисам-участникам дату нового дня, а те начисляют проценты и проводят платежи
 * через REST этого же сервиса. Дата сдвигается только после успешной обработки всеми участниками.
 */
@Service
@RequiredArgsConstructor
@Slf4j
public class BankDayService {

    private static final int MAX_DAYS = 3660;
    private static final DateTimeFormatter RU_DATE = DateTimeFormatter.ofPattern("dd.MM.yyyy");

    private final BankDayRepository days;
    private final BankProperties properties;
    private final DayCloseNotifier notifier;
    private final ReentrantLock closing = new ReentrantLock();

    public LocalDate current() {
        return days.findById(BankDay.ID).orElseThrow().getBankDate();
    }

    public DayCloseResult close(int count) {
        if (count < 1 || count > MAX_DAYS) {
            throw BankException.invalid("Количество дней должно быть от 1 до " + MAX_DAYS);
        }
        if (!closing.tryLock()) {
            throw new BankException(HttpStatus.CONFLICT, "DAY_CLOSE_IN_PROGRESS", "Закрытие дня уже выполняется");
        }
        try {
            List<String> events = new ArrayList<>();
            for (int i = 0; i < count; i++) {
                LocalDate next = current().plusDays(1);
                for (BankProperties.Participant participant : properties.dayClose().participants()) {
                    events.addAll(notify(participant, next));
                }
                BankDay day = days.findById(BankDay.ID).orElseThrow();
                day.setBankDate(next);
                days.saveAndFlush(day);
            }
            return new DayCloseResult(current(), events);
        } finally {
            closing.unlock();
        }
    }

    /**
     * Недоступный участник пропускается: его обработка устойчива к пропуску дней и наверстает их
     * при следующем закрытии. Ошибка обработки, напротив, останавливает закрытие — дата не меняется.
     */
    private List<String> notify(BankProperties.Participant participant, LocalDate date) {
        try {
            return notifier.dayOpened(participant.url(), date);
        } catch (ResourceAccessException e) {
            log.warn("Сервис «{0}» недоступен, день {1} обработан без него", participant.name(), date);
            return List.of(RU_DATE.format(date) + ": сервис «" + participant.name() + "» недоступен, обработка отложена");
        } catch (RestClientException e) {
            throw new BankException(HttpStatus.BAD_GATEWAY, "DAY_CLOSE_FAILED",
                "Сервис «" + participant.name() + "» не обработал день " + RU_DATE.format(date) + ", день не закрыт");
        }
    }
}

```

Листинг А.53 — account- service/src/main/java/by/bsuir/bank/account/service/DayCloseNotifier.java

```

package by.bsuir.bank.account.service;

import org.springframework.core.ParameterizedTypeReference;
import org.springframework.stereotype.Component;
import org.springframework.web.client.RestClient;

```

```

import java.time.LocalDate;
import java.util.List;
import java.util.Map;

/** REST-вызов сервиса-участника закрытия дня: POST {url}/api/internal/day-close. */
@Component
public class DayCloseNotifier {

    private final RestClient http;

    public DayCloseNotifier(RestClient.Builder builder) {
        this.http = builder.build();
    }

    /** Возвращает протокол операций, выполненных участником на дату нового банковского дня. */
    public List<String> dayOpened(String baseUrl, LocalDate date) {
        return http.post()
            .uri(baseUrl + "/api/internal/day-close")
            .body(Map.of("date", date.toString()))
            .retrieve()
            .body(new ParameterizedTypeReference<>() {
            });
    }
}

```

Листинг А.54 – account-service/src/main/java/by/bsuir/bank/account/service/LedgerInitializer.java

```

package by.bsuir.bank.account.service;

import by.bsuir.bank.account.BankProperties;
import by.bsuir.bank.account.domain.Side;
import by.bsuir.bank.account.dto.AccountView;
import by.bsuir.bank.account.dto.BatchRequest;
import by.bsuir.bank.account.dto.EntryRequest;
import by.bsuir.bank.account.dto.OpenAccountRequest;
import by.bsuir.bank.account.dto.OperationRequest;
import by.bsuir.bank.account.repository.CurrencyRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import org.springframework.stereotype.Component;

import java.math.BigDecimal;
import java.util.List;

/** При старте открывает собственные счета банка в каждой валюте: кассу и фонд развития со стартовым капиталом. */
@Component
@RequiredArgsConstructor
public class LedgerInitializer implements ApplicationRunner {

    public static final String CASH = "1010";
    public static final String FUND = "7327";

    private final LedgerService ledger;
    private final CurrencyRepository currencies;
    private final BankProperties properties;

    @Override
    public void run(ApplicationArguments args) {
        currencies.findAll().forEach(currency -> {
            String code = currency.getCode();
            ledger.open(new OpenAccountRequest(CASH, code, null, "Касса банка, " + code, null));
            AccountView fund = ledger.open(new OpenAccountRequest(FUND, code, null, "Фонд развития банка, " + code, null));

            BigDecimal capital = properties.fund().capital().get(code);
            if (capital != null && capital.signum() > 0) {
                ledger.post(new BatchRequest("FUND-CAPITAL-" + code, null, null, List.of(
                    new OperationRequest("Формирование фонда развития банка",
                        List.of(new EntryRequest(fund.number(), Side.CREDIT, capital))))));
            }
        });
    }
}

```

Листинг А.55 –

account-service/src/main/java/by/bsuir/bank/account/service/LedgerService.java

```
package by.bsuir.bank.account.service;

import by.bsuir.bank.account.domain.Account;
import by.bsuir.bank.account.domain.AccountNumbers;
import by.bsuir.bank.account.domain.Activity;
import by.bsuir.bank.account.domain.ChartAccount;
import by.bsuir.bank.account.domain.Operation;
import by.bsuir.bank.account.domain.PostingBatch;
import by.bsuir.bank.account.dto.AccountReport;
import by.bsuir.bank.account.dto.AccountView;
import by.bsuir.bank.account.dto.BatchRequest;
import by.bsuir.bank.account.dto.EntryRequest;
import by.bsuir.bank.account.dto.OpenAccountRequest;
import by.bsuir.bank.account.dto.OperationRequest;
import by.bsuir.bank.account.dto.OperationView;
import by.bsuir.bank.account.repository.AccountRepository;
import by.bsuir.bank.account.repository.ChartAccountRepository;
import by.bsuir.bank.account.repository.CurrencyRepository;
import by.bsuir.bank.account.repository.OperationRepository;
import by.bsuir.bank.account.repository.PostingBatchRepository;
import by.bsuir.bank.common.api.BankException;
import lombok.RequiredArgsConstructor;
import org.springframework.data.domain.PageRequest;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;

/** Главная книга: открытие счетов, проведение операций, отчёт о состоянии счетов. */
@Service
@RequiredArgsConstructor
public class LedgerService {

    public static final int BANK_OWNER = 0;

    private final AccountRepository accounts;
    private final ChartAccountRepository chart;
    private final CurrencyRepository currencies;
    private final OperationRepository operations;
    private final PostingBatchRepository batches;
    private final BankDayService bankDay;

    /**
     * Открывает лицевой счёт. Повторный запрос для того же договора и балансового счёта
     * возвращает уже открытый счёт — вызывающий сервис может безопасно повторять запрос.
     */
    @Transactional
    public AccountView open(OpenAccountRequest request) {
        ChartAccount chartAccount = chart.findById(request.chartCode())
            .orElseThrow(() -> BankException.invalid("Балансового счёта " + request.chartCode() + " нет в плане счетов"));
        if (!currencies.existsById(request.currency())) {
            throw BankException.invalid("Валюты " + request.currency() + " нет в справочнике");
        }
        if (request.contractRef() != null) {
            var existing = accounts.findByContractRefAndChartCode(request.contractRef(), request.chartCode());
            if (existing.isPresent()) {
                return AccountView.from(existing.get());
            }
        }
        int ownerCode = request.clientId() == null ? BANK_OWNER : request.clientId();
        int ownerSeq = accounts.maxOwnerSeq(ownerCode) + 1;

        Account account = new Account();
        account.setNumber(AccountNumbers.generate(chartAccount.getCode(), ownerCode, ownerSeq));
        account.setChart(chartAccount);
        account.setCurrency(request.currency());
        account.setName(request.name());
        account.setOwnerCode(ownerCode);
        account.setOwnerSeq(ownerSeq);
        account.setContractRef(request.contractRef());
    }
}
```

```

        account.setOpenedOn(bankDay.current());
        return AccountView.from(accounts.saveAndFlush(account));
    }

    /**
     * Проводит пакет операций в одной транзакции. После каждой операции остаток затронутых
     * счетов не должен стать отрицательным — иначе весь пакет откатывается.
     */
    @Transactional
    public List<OperationView> post(BatchRequest request) {
        if (batches.existsById(request.batchKey())) {
            return operations.findBatchKeyOrderBy(request.batchKey()).stream().map(OperationView::from).toList();
        }
        batches.saveAndFlush(new PostingBatch(request.batchKey()));

        LocalDate date = request.bankDate() != null ? request.bankDate() : bankDay.current();
        Map<String, Account> locked = new LinkedHashMap<>();
        List<Operation> posted = new ArrayList<>();
        for (OperationRequest item : request.operations()) {
            Operation operation = new Operation();
            operation.setBatchKey(request.batchKey());
            operation.setBankDate(date);
            operation.setDescription(item.description());
            operation.setContractRef(request.contractRef());
            operation.setCreatedAt(LocalDateTime.now());

            Set<Account> touched = new LinkedHashSet<>();
            String currency = null;
            for (EntryRequest entry : item.entries()) {
                Account account = locked.computeIfAbsent(entry.account(), number -> accounts.findNumberForUpdate(number)
                    .orElseThrow(() -> BankException.notFound("Счёт " + number + " не найден")));
                if (currency != null && !currency.equals(account.getCurrency())) {
                    throw BankException.invalid("Счета одной операции должны быть в одной валюте");
                }
                currency = account.getCurrency();
                account.apply(entry.side(), entry.amount());
                operation.addEntry(account, entry.side(), entry.amount());
                touched.add(account);
            }
            touched.forEach(account -> checkNotOverdrawn(account, item.description()));
            posted.add(operations.save(operation));
        }
        operations.flush();
        return posted.stream().map(OperationView::from).toList();
    }

    private static void checkNotOverdrawn(Account account, String operation) {
        if (account.getChart().getActivity() != Activity.ACTIVE_PASSIVE
            && account.getBalance().compareTo(BigDecimal.ZERO) < 0) {
            throw new BankException(HttpStatus.CONFLICT, "INSUFFICIENT_FUNDS",
                "Недостаточно средств на счёте " + account.getNumber() + " («" + account.getName()
                    + "») для операции «" + operation + "»");
        }
    }

    @Transactional(readOnly = true)
    public AccountView get(String number) {
        return accounts.findNumber(number).map(AccountView::from)
            .orElseThrow(() -> BankException.notFound("Счёт " + number + " не найден"));
    }

    /** Собственный счёт банка — касса (1010) или фонд развития (7327) — в заданной валюте. */
    @Transactional(readOnly = true)
    public AccountView system(String chartCode, String currency) {
        return accounts.findFirstByChartCodeAndCurrencyAndOwnerCodeAndContractRefIsNull(chartCode, currency, BANK_OWNER)
            .map(AccountView::from)
            .orElseThrow(() -> BankException.notFound(
                "Счёт банка " + chartCode + " в валюте " + currency + " не открыт"));
    }

    @Transactional(readOnly = true)
    public List<AccountView> find(Integer clientId, String contractRef) {
        List<Account> found;
        if (contractRef != null) {
            found = accounts.findContractRefOrderByNumber(contractRef);
        } else if (clientId != null) {
            found = accounts.findOwnerCodeOrderByNumber(clientId);
        } else {
            found = accounts.findAllByOrderByChartCodeAscNumberAsc();
        }
        return found.stream().map(AccountView::from).toList();
    }

```

```

@Transactional(readOnly = true)
public AccountReport report() {
    List<AccountView> all = find(null, null);
    Map<String, BigDecimal[]> sums = new LinkedHashMap<>();
    for (AccountView account : all) {
        BigDecimal[] sum = sums.computeIfAbsent(account.currency(), c -> new BigDecimal[]{BigDecimal.ZERO, BigDecimal.ZERO});
        sum[0] = sum[0].add(account.debit());
        sum[1] = sum[1].add(account.credit());
    }
    List<AccountReport.Total> totals = sums.entrySet().stream()
        .map(e -> new AccountReport.Total(e.getKey(), e.getValue()[0], e.getValue()[1]))
        .toList();
    return new AccountReport(bankDay.current(), all, totals);
}

@Transactional(readOnly = true)
public List<OperationView> journal(String contractRef, String account, int limit) {
    return operations.search(contractRef, account, PageRequest.of(0, limit)).stream()
        .map(OperationView::from)
        .toList();
}
}

```

Листинг А.56 – account-service/src/main/java/by/bsuir/bank/account/web/BankDayController.java

```

package by.bsuir.bank.account.web;

import by.bsuir.bank.account.dto.DayCloseResult;
import by.bsuir.bank.account.service.BankDayService;
import lombok.RequiredArgsConstructor;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import java.time.LocalDate;
import java.util.Map;

@RestController
@RequestMapping("/api/bank-day")
@RequiredArgsConstructor
public class BankDayController {

    private final BankDayService bankDay;

    @GetMapping
    public Map<String, LocalDate> current() {
        return Map.of("date", bankDay.current());
    }

    /** Процедура «Закрытие банковского дня»: закрывает указанное число дней подряд. */
    @PostMapping("/close")
    public DayCloseResult close(@RequestParam(defaultValue = "1") int days) {
        return bankDay.close(days);
    }
}

```

Листинг А.57 – account-service/src/main/java/by/bsuir/bank/account/web/LedgerController.java

```

package by.bsuir.bank.account.web;

import by.bsuir.bank.account.domain.ChartAccount;
import by.bsuir.bank.account.domain.Currency;
import by.bsuir.bank.account.dto.AccountReport;
import by.bsuir.bank.account.dto.AccountView;
import by.bsuir.bank.account.dto.BatchRequest;
import by.bsuir.bank.account.dto.OpenAccountRequest;
import by.bsuir.bank.account.dto.OperationView;
import by.bsuir.bank.account.repository.ChartAccountRepository;
import by.bsuir.bank.account.repository.CurrencyRepository;
import by.bsuir.bank.account.service.LedgerService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;

```

```

import org.springframework.data.domain.Sort;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
@RequestMapping("/api")
@RequiredArgsConstructor
public class LedgerController {

    private final LedgerService ledger;
    private final ChartAccountRepository chart;
    private final CurrencyRepository currencies;

    @GetMapping("/currencies")
    public List<Currency> currencies() {
        return currencies.findAll(Sort.by("code"));
    }

    @GetMapping("/chart-of-accounts")
    public List<ChartAccount> chartOfAccounts() {
        return chart.findAll(Sort.by("code"));
    }

    @GetMapping("/accounts")
    public List<AccountView> accounts(@RequestParam(required = false) Integer clientId,
                                     @RequestParam(required = false) String contractRef) {
        return ledger.find(clientId, contractRef);
    }

    @GetMapping("/accounts/system")
    public AccountView systemAccount(@RequestParam String chartCode, @RequestParam String currency) {
        return ledger.system(chartCode, currency);
    }

    @GetMapping("/accounts/{number}")
    public AccountView account(@PathVariable String number) {
        return ledger.get(number);
    }

    @PostMapping("/accounts")
    @ResponseStatus(HttpStatus.CREATED)
    public AccountView open(@Valid @RequestBody OpenAccountRequest request) {
        return ledger.open(request);
    }

    @PostMapping("/operations")
    @ResponseStatus(HttpStatus.CREATED)
    public List<OperationView> post(@Valid @RequestBody BatchRequest request) {
        return ledger.post(request);
    }

    @GetMapping("/operations")
    public List<OperationView> journal(@RequestParam(required = false) String contractRef,
                                     @RequestParam(required = false) String account,
                                     @RequestParam(defaultValue = "200") int limit) {
        return ledger.journal(contractRef, account, Math.min(limit, 1000));
    }

    @GetMapping("/report")
    public AccountReport report() {
        return ledger.report();
    }
}

```

Листинг А.58 – account-service/src/main/resources/static/index.html

```

<!doctype html>
<html lang="ru">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">

```

```

<title>Счета – Банк «Решение»</title>
<link rel="stylesheet" href="/ui/bank.css">
</head>
<body>
  <div id="app" v-cloak></div>

  <script src="/webjars/vue/dist/vue.global.prod.js"></script>
  <script src="/ui/bank.js"></script>
  <script src="/app.js"></script>
</body>
</html>

```

Листинг А.59 – account-service/src/main/resources/static/app.js

```

/* Web-клиент главной книги: отчёт о состоянии счетов, журнал проводок, план счетов, закрытие дня. */

const app = Vue.createApp({
  data() {
    return {
      tab: 'report', // report | journal | chart
      report: { bankDate: null, accounts: [], totals: [] },
      journal: [],
      chart: [],
      currency: 'BYN',
      events: [],
      error: '',
    };
  },
  computed: {
    currencies() {
      return [...new Set(this.report.accounts.map(a => a.currency))];
    },
    accounts() {
      return this.report.accounts.filter(a => a.currency === this.currency);
    },
    total() {
      return this.report.totals.find(t => t.currency === this.currency) || { debit: 0, credit: 0 };
    },
    fund() {
      return this.accounts.find(a => a.chartCode === '7327' && !a.contractRef);
    },
    cash() {
      return this.accounts.find(a => a.chartCode === '1010');
    },
  },
  async mounted() {
    try {
      this.chart = await Bank.api('GET', '/api/chart-of-accounts');
      await this.reload();
    } catch (e) {
      this.error = e.message;
    }
  },
  methods: {
    async reload() {
      this.report = await Bank.api('GET', '/api/report');
      this.journal = await Bank.api('GET', '/api/operations?limit=300');
    },
    async closed(result) {
      this.error = '';
      this.events = result.events;
      await this.reload();
    },
    date: Bank.date,
    money: Bank.money,
  },
  template: `
    <bank-nav active="accounts"></bank-nav>
    <main>
      <div class="toolbar">
        <div>
          <h1>Счета банка</h1>
          <p class="subtitle">Отчёт о состоянии счетов и процедура «Закрытие банковского дня»</p>
        </div>
        <bank-day @changed="closed" @failed="e => error = e.message"></bank-day>
      </div>

      <div v-if="error" id="error" class="alert error">{{ error }}</div>

      <div class="stats">

```

```

<div class="stat"><div class="label">Отчёт на дату</div><div class="value">{{ date(report.bankDate) }}</div></div>
<div class="stat"><div class="label">Счетов в отчёте</div><div class="value" id="accounts-count">{{ accounts.length }}</div></div>
<div class="stat"><div class="label">Фонд развития, {{ currency }}</div><div class="value">{{ fund ? money(fund.balance) : '-' }}</div></div>
<div class="stat"><div class="label">Касса, {{ currency }}</div><div class="value">{{ cash ? money(cash.balance) : '-' }}</div></div>

<div v-if="events.length" id="events" class="alert info">
  <b>Протокол закрытия дня</b>
  <div v-for="(event, i) in events.slice(-12)" :key="i">{{ event }}</div>
  <div v-if="events.length > 12" class="muted">... всего записей: {{ events.length }}</div>
</div>

<div class="toolbar">
  <div style="display: flex; gap: 6px">
    <button :class="{ primary: tab === 'report' }" @click="tab = 'report'">Отчёт по счетам</button>
    <button :class="{ primary: tab === 'journal' }" @click="tab = 'journal'">Журнал проводок</button>
    <button :class="{ primary: tab === 'chart' }" @click="tab = 'chart'">План счетов</button>
  </div>
  <div v-if="tab === 'report'" style="display: flex; gap: 6px">
    <button v-for="c in currencies" :key="c" class="small" :class="{ primary: c === currency }" @click="currency = c">{{ c }}</button>
  </div>
</div>

<div v-if="tab === 'report'" class="card">
  <table id="report">
    <thead>
      <tr>
        <th>Номер счёта</th><th>Код</th><th>Название счёта</th><th>Активность</th><th>Договор</th>
      </tr>
    </thead>
    <tbody>
      <tr v-for="a in accounts" :key="a.number">
        <td class="mono">{{ a.number }}</td>
        <td class="mono">{{ a.chartCode }}</td>
        <td>{{ a.name }}<div class="muted" style="font-size: 12.5px">{{ a.chartName }}</div></td>
        <td><span class="badge" :class="{ ok: a.activity === 'ACTIVE' }">{{ a.activityTitle }}</span></td>
        <td class="mono">{{ a.contractRef }}</td>
        <td class="num">{{ money(a.debit) }}</td>
        <td class="num">{{ money(a.credit) }}</td>
        <td class="num"><b>{{ money(a.balance) }}</b></td>
      </tr>
    </tbody>
    <tfoot>
      <tr>
        <td colspan="5">Итого обороты, {{ currency }}</td>
        <td class="num">{{ money(total.debit) }}</td>
        <td class="num">{{ money(total.credit) }}</td>
        <td></td>
      </tr>
    </tfoot>
  </table>
</div>

<div v-if="tab === 'journal'" class="card">
  <table id="journal">
    <thead>
      <tr><th>№</th><th>Дата</th><th>Операция</th><th>Договор</th><th>Счёт</th><th class="num">Дебет</th><th class="num">Кредит</th></tr>
    </thead>
    <tbody>
      <template v-for="op in journal" :key="op.id">
        <tr v-for="(entry, i) in op.entries" :key="op.id + '-' + i">
          <td>{{ i === 0 ? op.id : '' }}</td>
          <td>{{ i === 0 ? date(op.bankDate) : '' }}</td>
          <td>{{ i === 0 ? op.description : '' }}</td>
          <td class="mono">{{ i === 0 ? op.contractRef : '' }}</td>
          <td class="mono">{{ entry.account }} <span class="muted">{{ entry.accountName }}</span></td>
          <td class="num">{{ entry.side === 'DEBIT' ? money(entry.amount) : '' }}</td>
          <td class="num">{{ entry.side === 'CREDIT' ? money(entry.amount) : '' }}</td>
        </tr>
      </template>
    </tbody>
  </table>
  <div v-if="!journal.length" class="empty">Проводок пока нет</div>
</div>

<div v-if="tab === 'chart'" class="card">

```

```

<table id="chart">
  <thead><tr><th>Балансовый счёт</th><th>Наименование</th><th>Активность</th></tr></thead>
  <tbody>
    <tr v-for="c in chart" :key="c.code">
      <td class="mono">{{ c.code }}</td>
      <td>{{ c.name }}</td>
      <td>{{ { ACTIVE: 'Активный', PASSIVE: 'Пассивный', ACTIVE_PASSIVE: 'Активно-пассивный' }[c.activity] }}</td>
    </tr>
  </tbody>
</table>
</div>
</main>`,
});

app.component('bank-nav', Bank.navComponent);
app.component('bank-day', Bank.dayComponent);
app.directive('mask', Bank.maskDirective);
app.mount('#app');

```

Листинг А.60 –

account-service/src/test/java/by/bsuir/bank/account/AccountNumbersTest.java

```

package by.bsuir.bank.account;

import by.bsuir.bank.account.domain.AccountNumbers;
import by.bsuir.bank.account.domain.Activity;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;

import java.math.BigDecimal;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;

/** Модульные тесты правил формирования номера счёта и расчёта сальдо. */
class AccountNumbersTest {

    @ParameterizedTest
    @CsvSource({
        "1010, 0, 1, 1010000000019",
        "7327, 0, 2, 7327000000029",
        "3014, 4, 1, 301400004001",
        "2400, 12345, 17, 240012345017",
    })
    void numberConsistsOfBalanceAccountClientCodeSequenceAndKey(String chart, int client, int seq, String prefix) {
        String number = AccountNumbers.generate(chart, client, seq);

        assertThat(number).hasSize(13).containsOnlyDigits().startsWith(prefix.substring(0, 12));
        assertThat(number.substring(0, 4)).isEqualTo(chart);
        assertThat(Integer.parseInt(number.substring(4, 9))).isEqualTo(client);
        assertThat(Integer.parseInt(number.substring(9, 12))).isEqualTo(seq);
        assertThat(AccountNumbers.isValid(number)).isTrue();
    }

    @Test
    void checkDigitDetectsSingleDigitError() {
        String number = AccountNumbers.generate("3014", 4, 1);
        for (int position = 0; position < 12; position++) {
            char wrong = number.charAt(position) == '9' ? '0' : (char) (number.charAt(position) + 1);
            String corrupted = number.substring(0, position) + wrong + number.substring(position + 1);
            assertThat(AccountNumbers.isValid(corrupted)).as("ошибка в разряде %d", position + 1).isFalse();
        }
    }

    @Test
    void rejectsMalformedNumbers() {
        assertThat(AccountNumbers.isValid(null)).isFalse();
        assertThat(AccountNumbers.isValid("30140000400")).isFalse();
        assertThat(AccountNumbers.isValid("30140000400A1")).isFalse();
        assertThatThrownBy(() -> AccountNumbers.generate("3014", 100_000, 1)).isInstanceOf(IllegalArgumentException.class);
        assertThatThrownBy(() -> AccountNumbers.generate("3014", 1, 1000)).isInstanceOf(IllegalArgumentException.class);
        assertThatThrownBy(() -> AccountNumbers.generate("30", 1, 1)).isInstanceOf(IllegalArgumentException.class);
    }

    @Test
    void balanceDependsOnActivity() {
        BigDecimal debit = new BigDecimal("700.00");
        BigDecimal credit = new BigDecimal("1000.00");
    }

```

```

        assertThat(Activity.ACTIVE.balance(debit, credit)).isEqualByComparingTo("-300.00");
        assertThat(Activity.PASSIVE.balance(debit, credit)).isEqualByComparingTo("300.00");
        assertThat(Activity.ACTIVE_PASSIVE.balance(debit, credit)).isEqualByComparingTo("-300.00");
    }
}

```

Листинг А.61 –

account-service/src/test/java/by/bsuir/bank/account/BankDayIT.java

```

package by.bsuir.bank.account;

import by.bsuir.bank.account.service.BankDayService;
import by.bsuir.bank.account.service.DayCloseNotifier;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.HttpStatus;
import org.springframework.http.MediaType;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.web.client.HttpServerErrorException;
import org.springframework.web.client.ResourceAccessException;

import java.time.LocalDate;
import java.util.List;

import static org.assertj.core.api.Assertions.assertThat;
import static org.hamcrest.Matchers.containsString;
import static org.hamcrest.Matchers.hasItem;
import static org.hamcrest.Matchers.hasSize;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.ArgumentMatchers.anyString;
import static org.mockito.ArgumentMatchers.contains;
import static org.mockito.ArgumentMatchers.eq;
import static org.mockito.Mockito.times;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

/** Интеграционные тесты REST и процедуры закрытия дня; сервисы-участники заменены заглушкой. */
@SpringBootTest(properties = "spring.datasource.url=jdbc:h2:mem:${random.uuid};DB_CLOSE_DELAY=-1")
@AutoConfigureMockMvc
class BankDayIT {

    @Autowired
    private MockMvc mvc;
    @Autowired
    private BankDayService bankDay;
    @MockitoBean
    private DayCloseNotifier notifier;

    @Test
    @DisplayName("Закрытие дня объявляет новую дату каждому участнику и сдвигает банковский день")
    void closesSeveralDays() throws Exception {
        LocalDate start = bankDay.current();
        when(notifier.dayOpened(anyString(), any())).thenReturn(List.of("Начислены проценты"));

        mvc.perform(post("/api/bank-day/close").param("days", "3"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.bankDate").value(start.plusDays(3).toString()))
            .andExpect(jsonPath("$.events", hasSize(6)));

        verify(notifier).dayOpened(contains("8083"), eq(start.plusDays(1)));
        verify(notifier).dayOpened(contains("8084"), eq(start.plusDays(3)));
        verify(notifier, times(6)).dayOpened(anyString(), any());
        mvc.perform(get("/api/bank-day")).andExpect(jsonPath("$.date").value(start.plusDays(3).toString()));
    }

    @Test
    @DisplayName("Недоступный участник пропускается, день закрывается")
    void unavailableParticipantIsSkipped() throws Exception {
        LocalDate start = bankDay.current();
        when(notifier.dayOpened(anyString(), any())).thenThrow(new ResourceAccessException("Connection refused"));

        mvc.perform(post("/api/bank-day/close"))

```

```

        .andExpect(status().isOk())
        .andExpect(jsonPath("$.bankDate").value(start.plusDays(1).toString()))
        .andExpect(jsonPath("$.events", hasItem(containsString("недоступен"))));
    }

    @Test
    @DisplayName("Ошибка участника останавливает закрытие: дата не меняется")
    void participantFailureKeepsDayOpen() throws Exception {
        LocalDate start = bankDay.current();
        when(notifier.dayOpened(anyString(), any())).thenThrow(new HttpServerErrorException(HttpStatus.INTERNAL_SERVER_ERROR));

        mvc.perform(post("/api/bank-day/close"))
            .andExpect(status().isBadGateway())
            .andExpect(jsonPath("$.code").value("DAY_CLOSE_FAILED"));

        assertThat(bankDay.current()).isEqualTo(start);
    }

    @Test
    @DisplayName("Количество дней проверяется")
    void rejectsWrongDaysCount() throws Exception {
        mvc.perform(post("/api/bank-day/close").param("days", "0")).andExpect(status().isBadRequest());
    }

    @Test
    @DisplayName("Справочники «План счетов» и валют, отчёт по счетам")
    void dictionariesAndReport() throws Exception {
        mvc.perform(get("/api/chart-of-accounts"))
            .andExpect(jsonPath("$[?(@.code == '7327')].activity").value("PASSIVE"))
            .andExpect(jsonPath("$[?(@.code == '1010')].activity").value("ACTIVE"));
        mvc.perform(get("/api/currencies")).andExpect(jsonPath("$[*].code", hasItem("BYN")));
        mvc.perform(get("/api/report"))
            .andExpect(jsonPath("$.accounts", hasSize(6)))
            .andExpect(jsonPath("$.totals[?(@.currency == 'BYN')].credit").value(100000000000.0));
    }

    @Test
    @DisplayName("Запрос на проводку проверяется: сумма положительна, счёт существует")
    void validatesPostingRequest() throws Exception {
        String negative = """
            {"batchKey": "K1", "operations": [{"description": "Тест",
            "entries": [{"account": "1010000000019", "side": "DEBIT", "amount": -5}]}]}""";
        mvc.perform(post("/api/operations").contentType(MediaType.APPLICATION_JSON).content(negative))
            .andExpect(status().isBadRequest());

        String unknown = """
            {"batchKey": "K2", "operations": [{"description": "Тест",
            "entries": [{"account": "000000000000", "side": "DEBIT", "amount": 5}]}]}""";
        mvc.perform(post("/api/operations").contentType(MediaType.APPLICATION_JSON).content(unknown))
            .andExpect(status().isNotFound());
    }
}

```

Листинг А.62 – account-service/src/test/java/by/bsuir/bank/account/LedgerServiceIT.java

```

package by.bsuir.bank.account;

import by.bsuir.bank.account.domain.AccountNumbers;
import by.bsuir.bank.account.domain.Side;
import by.bsuir.bank.account.dto.AccountView;
import by.bsuir.bank.account.dto.BatchRequest;
import by.bsuir.bank.account.dto.EntryRequest;
import by.bsuir.bank.account.dto.OpenAccountRequest;
import by.bsuir.bank.account.dto.OperationRequest;
import by.bsuir.bank.common.api.BankException;
import by.bsuir.bank.account.service.LedgerService;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.jdbc.core.JdbcTemplate;

import java.math.BigDecimal;
import java.util.List;
import java.util.UUID;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;

```

```

/** Интеграционные тесты главной книги на H2: счета, проводки, атомарность и идемпотентность. */
@SpringBootTest(properties = {
    "spring.datasource.url=jdbc:h2:mem:${random.uuid};DB_CLOSE_DELAY=-1",
    "bank.fund.capital.BYN=1000000"})
class LedgerServiceIT {

    @Autowired
    private LedgerService ledger;
    @Autowired
    private JdbcTemplate jdbc;

    @Test
    @DisplayName("При старте открыты касса и фонд развития со стартовым капиталом")
    void systemAccountsAreOpenedOnStartup() {
        AccountView cash = ledger.system("1010", "BYN");
        AccountView fund = ledger.system("7327", "BYN");

        assertThat(cash.activity()).isEqualTo("ACTIVE");
        assertThat(fund.activity()).isEqualTo("PASSIVE");
        assertThat(fund.credit()).isEqualToByComparingTo("1000000.00");
        assertThat(fund.balance()).isEqualToByComparingTo("1000000.00");
        assertThat(ledger.report().accounts()).hasSizeGreaterThanOrEqualTo(6);
    }

    @Test
    @DisplayName("Номер нового счёта: балансовый счёт, код клиента, порядковый номер, ключ")
    void opensClientAccounts() {
        AccountView main = ledger.open(new OpenAccountRequest("3404", "BYN", 41, "Иванов Иван Иванович", "D-41"));
        AccountView interest = ledger.open(new OpenAccountRequest("3470", "BYN", 41, "Иванов Иван Иванович", "D-41"));

        assertThat(main.number()).startsWith("340400041001");
        assertThat(interest.number()).startsWith("347000041002");
        assertThat(AccountNumbers.isValid(main.number())).isTrue();
        assertThat(main.balance()).isEqualToByComparingTo("0");
        assertThat(ledger.find(41, null)).hasSize(2);
        assertThat(ledger.find(null, "D-41").extracting(AccountView::chartCode).containsExactly("3404", "3470"));
    }

    @Test
    @DisplayName("Повторное открытие счёта по тому же договору возвращает существующий счёт")
    void openingIsIdempotentPerContract() {
        OpenAccountRequest request = new OpenAccountRequest("3414", "BYN", 42, "Петров Пётр Петрович", "D-42");

        assertThat(ledger.open(request).number()).isEqualTo(ledger.open(request).number());
        assertThat(ledger.find(42, null)).hasSize(1);
    }

    @Test
    @DisplayName("Счёт нельзя открыть по коду, которого нет в плане счетов, или в неизвестной валюте")
    void rejectsUnknownChartCodeAndCurrency() {
        assertThatThrownBy(() -> ledger.open(new OpenAccountRequest("9999", "BYN", 1, "x", null)))
            .isInstanceOf(BankException.class).hasMessageContaining("нет в плане счетов");
        assertThatThrownBy(() -> ledger.open(new OpenAccountRequest("3014", "RUB", 1, "x", null)))
            .isInstanceOf(BankException.class).hasMessageContaining("нет в справочнике");
    }

    @Test
    @DisplayName("Проводка меняет обороты; сальдо активного счёта растёт по дебету, пассивного — по кредиту")
    void postingChangesTurnovers() {
        AccountView cash = ledger.system("1010", "USD");
        AccountView client = ledger.open(new OpenAccountRequest("3014", "USD", 43, "Сидоров", "T-43"));

        ledger.post(batch(
            operation("Внесение денег в кассу", entry(cash, Side.DEBIT, "500")),
            operation("Перевод денег с кассы на текущий счёт", entry(cash, Side.CREDIT, "500"), entry(client, Side.CREDIT, "500"))));

        assertThat(ledger.get(cash.number()).satisfies(a -> {
            assertThat(a.debit()).isEqualToByComparingTo("500");
            assertThat(a.credit()).isEqualToByComparingTo("500");
            assertThat(a.balance()).isEqualToByComparingTo("0");
        }));
        assertThat(ledger.get(client.number()).balance()).isEqualToByComparingTo("500");
        assertThat(ledger.journal("T-43", null, 10)).hasSize(2);
    }

    @Test
    @DisplayName("При нехватке средств откатывается весь пакет операций")
    void batchIsAtomic() {
        AccountView cash = ledger.system("1010", "EUR");
        AccountView client = ledger.open(new OpenAccountRequest("3014", "EUR", 44, "Кузнецов", "T-44"));
        String key = UUID.randomUUID().toString();
    }
}

```

```

        assertThatThrownBy(() -> ledger.post(new BatchRequest(key, "T-44", null, List.of(
            operation("Внесение денег в кассу", entry(cash, Side.DEBIT, "100")),
            operation("Списание со счёта клиента", entry(client, Side.DEBIT, "1"), entry(cash, Side.DEBIT, "1")))))
            .isInstanceOfSatisfying(BankException.class, e -> assertThat(e.getCode()).isEqualTo("INSUFFICIENT_FUNDS"));

        assertThat(ledger.get(cash.number()).debit()).isEqualByComparingTo("0");
        assertThat(ledger.journal("T-44", null, 10)).isEmpty();
        assertThat(jdbc.queryForObject("select count(*) from posting_batch where batch_key = ?", Integer.class, key)).isZero();
    }

    @Test
    @DisplayName("Повтор пакета с тем же ключом не проводит операции второй раз")
    void postingIsIdempotent() {
        AccountView cash = ledger.system("1010", "BYN");
        BigDecimal before = ledger.get(cash.number()).debit();
        BatchRequest request = batch(operation("Внесение денег в кассу", entry(cash, Side.DEBIT, "250")));

        var first = ledger.post(request);
        var second = ledger.post(request);

        assertThat(second).extracting("id").isEqualTo(first.stream().map(o -> o.id()).toList());
        assertThat(ledger.get(cash.number()).debit()).isEqualByComparingTo(before.add(new BigDecimal("250")));
    }

    @Test
    @DisplayName("В одной операции нельзя смешивать счета разных валют")
    void rejectsMixedCurrencies() {
        AccountView byn = ledger.system("1010", "BYN");
        AccountView usd = ledger.system("1010", "USD");

        assertThatThrownBy(() -> ledger.post(batch(
            operation("Перевод", entry(byn, Side.DEBIT, "10"), entry(usd, Side.DEBIT, "10"))))
            .isInstanceOf(BankException.class).hasMessageContaining("в одной валюте"));
    }

    private static BatchRequest batch(OperationRequest... operations) {
        return new BatchRequest(UUID.randomUUID().toString(), "T-43", null, List.of(operations));
    }

    private static OperationRequest operation(String description, EntryRequest... entries) {
        return new OperationRequest(description, List.of(entries));
    }

    private static EntryRequest entry(AccountView account, Side side, String amount) {
        return new EntryRequest(account.number(), side, new BigDecimal(amount));
    }
}

```

Микросервис «Депозиты» (deposit-service)

Листинг A.63 – deposit-service/pom.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>by.bsuir.bank</groupId>
        <artifactId>bank</artifactId>
        <version>1.0.0</version>
    </parent>

    <artifactId>deposit-service</artifactId>
    <name>deposit-service</name>
    <description>ЛР2. Микросервис депозитных операций с физическими лицами</description>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
    </dependencies>

```

```

</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
<dependency>
  <groupId>by.bsuir.bank</groupId>
  <artifactId>bank-common</artifactId>
</dependency>
<dependency>
  <groupId>by.bsuir.bank</groupId>
  <artifactId>ui-kit</artifactId>
</dependency>
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>by.bsuir.bank</groupId>
  <artifactId>bank-common</artifactId>
  <version>${project.version}</version>
  <type>test-jar</type>
  <scope>test</scope>
</dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-failsafe-plugin</artifactId>
    </plugin>
  </plugins>
</build>
</project>

```

Листинг А.64 – deposit-service/src/main/resources/application.yml

```

server:
  port: ${DEPOSIT_SERVICE_PORT:8083}

spring:
  application:
    name: deposit-service
  datasource:
    url: jdbc:h2:mem:deposits;DB_CLOSE_DELAY=-1
    username: sa
  sql:
    init:
      mode: always
  jpa:
    hibernate:
      ddl-auto: validate
      open-in-view: false
  h2:
    console:
      enabled: true
  http:
    client:
      connect-timeout: 2s
      read-timeout: 120s

bank:
  services:

```

```
clients: ${CLIENT_SERVICE_URL:http://localhost:8081}
accounts: ${ACCOUNT_SERVICE_URL:http://localhost:8082}
```

Листинг А.65 – deposit-service/src/main/resources/schema.sql

```
-- Схема данных сервиса депозитов

-- Справочник депозитных программ («вид депозита»)
create table deposit_product (
    id                bigint generated by default as identity primary key,
    name              varchar(100) not null,
    kind              varchar(12)  not null check (kind in ('REVOCABLE', 'IRREVOCABLE')),
    currency           varchar(3)  not null,
    rate              decimal(6, 2) not null,
    min_amount        decimal(19, 2) not null,
    min_term_months   int          not null,
    max_term_months   int          not null,
    description       varchar(300) not null
);

create table deposit_contract (
    id                bigint generated by default as identity primary key,
    number            varchar(20) not null unique,
    product_id        bigint      not null references deposit_product,
    client_id         bigint      not null,                -- клиент из модуля «Клиенты» (ЛР1)
    client_name       varchar(200) not null,
    currency           varchar(3)  not null,
    amount            decimal(19, 2) not null,
    rate              decimal(6, 2) not null,
    term_months       int          not null,
    start_date        date         not null,
    end_date          date         not null,
    main_account       varchar(13) not null,                -- счёт основной суммы вклада
    interest_account   varchar(13) not null,                -- счёт процентов по вкладу
    status             varchar(8)  not null check (status in ('ACTIVE', 'CLOSED')),
    accrued            decimal(19, 2) not null default 0,    -- начислено процентов
    paid              decimal(19, 2) not null default 0,    -- выплачено процентов
    paid_months       int          not null default 0,      -- оплачено полных месяцев
    closed_on         date
);

create index ix_deposit_client on deposit_contract (client_id);
```

Листинг А.66 – deposit-service/src/main/resources/data.sql

```
-- Депозитные программы банка «Решение» (вариант 4): условия приближены к действующим на gbank.by
insert into deposit_product (name, kind, currency, rate, min_amount, min_term_months, max_term_months, description)
values ('Свободное накопление', 'REVOCABLE', 'BYN', 7.00, 100.00, 1, 12,
    'Отзывный вклад до востребования, проценты выплачиваются ежемесячно'),
    ('Online-решение', 'IRREVOCABLE', 'BYN', 12.90, 500.00, 3, 13,
    'Срочный безотзывный вклад, проценты выплачиваются в конце срока'),
    ('Свободное накопление', 'REVOCABLE', 'USD', 1.50, 100.00, 1, 12,
    'Отзывный вклад до востребования, проценты выплачиваются ежемесячно'),
    ('Online-решение', 'IRREVOCABLE', 'USD', 3.00, 500.00, 3, 13,
    'Срочный безотзывный вклад, проценты выплачиваются в конце срока'),
    ('Online-решение', 'IRREVOCABLE', 'EUR', 2.00, 500.00, 3, 13,
    'Срочный безотзывный вклад, проценты выплачиваются в конце срока');
```

Листинг А.67 – deposit-service/src/main/java/by/bsuir/bank/deposit/DepositServiceApplication.java

```
package by.bsuir.bank.deposit;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

/** Вместе с пакетом сервиса сканируется общая библиотека: REST-клиенты смежных сервисов и обработчик ошибок. */
@SpringBootApplication(scanBasePackages = {"by.bsuir.bank.deposit", "by.bsuir.bank.common"})
public class DepositServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(DepositServiceApplication.class, args);
    }
}
```

Листинг А.68 – deposit-service/src/main/java/by/bsuir/bank/deposit/domain/DepositContract.java

```
package by.bsuir.bank.deposit.domain;

import jakarta.persistence.Entity;
import jakarta.persistence.EnumType;
import jakarta.persistence.Enumerated;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.ManyToOne;
import lombok.Getter;
import lombok.Setter;

import java.math.BigDecimal;
import java.time.LocalDate;

/** Депозитный договор с физическим лицом. */
@Entity
@Getter
@Setter
public class DepositContract {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String number;

    @ManyToOne(fetch = FetchType.EAGER, optional = false)
    private DepositProduct product;

    private Long clientId;
    private String clientName;
    private String currency;
    private BigDecimal amount;
    private BigDecimal rate;
    private int termMonths;
    private LocalDate startDate;
    private LocalDate endDate;
    private String mainAccount;
    private String interestAccount;

    @Enumerated(EnumType.STRING)
    private DepositStatus status;

    private BigDecimal accrued = BigDecimal.ZERO;
    private BigDecimal paid = BigDecimal.ZERO;
    private int paidMonths;
    private LocalDate closedOn;

    public boolean isActive() {
        return status == DepositStatus.ACTIVE;
    }
}
```

Листинг А.69 – deposit-service/src/main/java/by/bsuir/bank/deposit/domain/DepositKind.java

```
package by.bsuir.bank.deposit.domain;

import lombok.Getter;
import lombok.RequiredArgsConstructor;

/** Вид вклада определяет порядок выплаты процентов и балансовые счета из плана счетов. */
@Getter
@RequiredArgsConstructor
public enum DepositKind {

    /** Отзывный, до востребования: проценты выплачиваются ежемесячно, вклад можно забрать досрочно. */
    REVOCABLE("Отзывный, до востребования", "3404", "3470"),

    /** Срочный безотзывный: проценты выплачиваются в конце срока, досрочный отзыв невозможен. */
    IRREVOCABLE("Срочный безотзывный", "3414", "3471");

    private final String title;
    private final String mainChartCode;
    private final String interestChartCode;
```

```
}
```

Листинг А.70 – deposit-service/src/main/java/by/bsuir/bank/deposit/domain/DepositProduct.java

```
package by.bsuir.bank.deposit.domain;

import jakarta.persistence.Entity;
import jakarta.persistence.EnumType;
import jakarta.persistence.Enumerated;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import lombok.Getter;
import lombok.Setter;

import java.math.BigDecimal;

/** Депозитная программа – запись справочника «вид депозита». */
@Entity
@Getter
@Setter
public class DepositProduct {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @Enumerated(EnumType.STRING)
    private DepositKind kind;

    private String currency;
    private BigDecimal rate;
    private BigDecimal minAmount;
    private int minTermMonths;
    private int maxTermMonths;
    private String description;

    public String getKindTitle() {
        return kind.getTitle();
    }
}
```

Листинг А.71 – deposit-service/src/main/java/by/bsuir/bank/deposit/domain/DepositStatus.java

```
package by.bsuir.bank.deposit.domain;

public enum DepositStatus {
    ACTIVE, CLOSED
}
```

Листинг А.72 – deposit- service/src/main/java/by/bsuir/bank/deposit/repository/DepositContractRepository.java

```
package by.bsuir.bank.deposit.repository;

import by.bsuir.bank.deposit.domain.DepositContract;
import by.bsuir.bank.deposit.domain.DepositStatus;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;

import java.util.List;

public interface DepositContractRepository extends JpaRepository<DepositContract, Long> {

    boolean existsByNumber(String number);

    List<DepositContract> findAllByOrderByIdDesc();

    List<DepositContract> findByClientIdOrderById(Long clientId);

    @Query("select c.id from DepositContract c where c.status = :status order by c.id")
}
```

```

    List<Long> findIdsByStatus(DepositStatus status);

    @Query("select max(c.number) from DepositContract c")
    String maxNumber();
}

```

Листинг А.73 – deposit-service/src/main/java/by/bsuir/bank/deposit/repository/DepositProductRepository.java

```

package by.bsuir.bank.deposit.repository;

import by.bsuir.bank.deposit.domain.DepositProduct;
import org.springframework.data.jpa.repository.JpaRepository;

public interface DepositProductRepository extends JpaRepository<DepositProduct, Long> {
}

```

Листинг А.74 – deposit-service/src/main/java/by/bsuir/bank/deposit/dto/DepositDetails.java

```

package by.bsuir.bank.deposit.dto;

import by.bsuir.bank.common.ledger.AccountInfo;
import by.bsuir.bank.common.ledger.OperationInfo;

import java.util.List;

/** Договор вместе с состоянием его счетов и проводками из главной книги. */
public record DepositDetails(DepositView contract, List<AccountInfo> accounts, List<OperationInfo> operations) {
}

```

Листинг А.75 – deposit-service/src/main/java/by/bsuir/bank/deposit/dto/DepositRequest.java

```

package by.bsuir.bank.deposit.dto;

import jakarta.validation.constraints.Digits;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Positive;

import java.math.BigDecimal;
import java.time.LocalDate;

import static by.bsuir.bank.common.api.CommonExceptionHandler.REQUIRED;

/** Данные формы заключения депозитного договора. */
public record DepositRequest(

    @NotBlank(message = REQUIRED)
    @Pattern(regexp = NUMBER_FORMAT, message = "Формат номера договора: Д-000001")
    String number,

    @NotNull(message = REQUIRED)
    Long productId,

    @NotBlank(message = REQUIRED)
    String currency,

    @NotNull(message = REQUIRED)
    Long clientId,

    @NotNull(message = REQUIRED)
    @Positive(message = "Сумма должна быть больше нуля")
    @Digits(integer = 12, fraction = 2, message = "Денежная сумма: не более 2 знаков после запятой")
    BigDecimal amount,

    @NotNull(message = REQUIRED)
    BigDecimal rate,

    @NotNull(message = REQUIRED)
    @Positive(message = "Срок должен быть больше нуля")
    Integer termMonths,
)

```

```

        @NotNull(message = REQUIRED)
        LocalDate startDate,

        @NotNull(message = REQUIRED)
        LocalDate endDate
    ) {

        public static final String NUMBER_FORMAT = "^Д-\\d{6}$";
    }

```

Листинг А.76 – deposit-service/src/main/java/by/bsuir/bank/deposit/dto/DepositView.java

```

package by.bsuir.bank.deposit.dto;

import by.bsuir.bank.deposit.domain.DepositContract;

import java.math.BigDecimal;
import java.time.LocalDate;

public record DepositView(
    Long id,
    String number,
    Long productId,
    String productName,
    String kind,
    String kindTitle,
    Long clientId,
    String clientName,
    String currency,
    BigDecimal amount,
    BigDecimal rate,
    int termMonths,
    LocalDate startDate,
    LocalDate endDate,
    String mainAccount,
    String interestAccount,
    String status,
    BigDecimal accrued,
    BigDecimal paid,
    LocalDate closedOn
) {

    public static DepositView from(DepositContract c) {
        return new DepositView(c.getId(), c.getNumber(), c.getProduct().getId(), c.getProduct().getName(),
            c.getProduct().getKind().name(), c.getProduct().getKind().getTitle(), c.getClientId(),
            c.getClientName(), c.getCurrency(), c.getAmount(), c.getRate(), c.getTermMonths(),
            c.getStartDate(), c.getEndDate(), c.getMainAccount(), c.getInterestAccount(),
            c.getStatus().name(), c.getAccrued(), c.getPaid(), c.getClosedOn());
    }
}

```

Листинг А.77 – deposit-service/src/main/java/by/bsuir/bank/deposit/dto/Meta.java

```

package by.bsuir.bank.deposit.dto;

import by.bsuir.bank.common.ledger.CurrencyInfo;

import java.time.LocalDate;
import java.util.List;

/** Данные для формы договора: банковский день, справочник валют, следующий свободный номер договора. */
public record Meta(LocalDate bankDate, List<CurrencyInfo> currencies, String nextNumber) {
}

```

Листинг А.78 – deposit- service/src/main/java/by/bsuir/bank/deposit/service/DayCloseProcessor.java

```

package by.bsuir.bank.deposit.service;

import by.bsuir.bank.deposit.domain.DepositStatus;
import by.bsuir.bank.deposit.repository.DepositContractRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Component;

```

```

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

/** Участие сервиса депозитов в процедуре «Закрытие банковского дня»: каждый договор — в своей транзакции. */
@Component
@RequiredArgsConstructor
public class DayCloseProcessor {

    private final DepositContractRepository contracts;
    private final DepositService deposits;

    public List<String> process(LocalDate date) {
        List<String> events = new ArrayList<>();
        for (Long id : contracts.findIdsByStatus(DepositStatus.ACTIVE)) {
            events.addAll(deposits.processDay(id, date));
        }
        return events;
    }
}

```

Листинг А.79 —

deposit-service/src/main/java/by/bsuir/bank/deposit/service/DepositPostings.java

```

package by.bsuir.bank.deposit.service;

import by.bsuir.bank.common.ledger.Entry;
import by.bsuir.bank.common.ledger.Posting;

import java.math.BigDecimal;
import java.util.List;

/**
 * Проводки процесса «Депозитная программа» — в точности по упрощённой схеме из задания.
 * Касса — активный счёт; фонд развития, счёт вклада и процентный счёт — пассивные.
 */
public final class DepositPostings {

    /** Заключение договора: деньги вносятся в кассу, зачисляются на счёт вклада и передаются в фонд развития. */
    public static List<Posting> open(String cash, String fund, String main, BigDecimal amount) {
        return List.of(
            Posting.of("Внесение денег в кассу", Entry.debit(cash, amount)),
            Posting.of("Перевод денег с кассы на текущий счёт", Entry.credit(cash, amount), Entry.credit(main, amount)),
            Posting.of("Использование денег банком", Entry.debit(main, amount), Entry.credit(fund, amount));
        )
    }

    public static Posting accrual(String fund, String interest, BigDecimal amount) {
        return Posting.of("Начисление процентов по депозиту", Entry.debit(fund, amount), Entry.credit(interest, amount));
    }

    /** Выплата процентов: с процентного счёта в кассу и из кассы клиенту. */
    public static List<Posting> payout(String cash, String interest, BigDecimal amount) {
        return List.of(
            Posting.of("Перевод процентов в кассу", Entry.debit(interest, amount), Entry.debit(cash, amount)),
            Posting.of("Вывод процентов из кассы", Entry.credit(cash, amount));
        )
    }

    /** Окончание депозита: сумма возвращается из фонда на счёт вклада, затем через кассу клиенту. */
    public static List<Posting> close(String cash, String fund, String main, BigDecimal amount) {
        return List.of(
            Posting.of("Окончание депозита", Entry.debit(fund, amount), Entry.credit(main, amount)),
            Posting.of("Перевод депозита в кассу", Entry.debit(main, amount), Entry.debit(cash, amount)),
            Posting.of("Вывод денег из кассы", Entry.credit(cash, amount));
        )
    }

    private DepositPostings() {
    }
}

```

Листинг А.80 —

deposit-service/src/main/java/by/bsuir/bank/deposit/service/DepositService.java

```

package by.bsuir.bank.deposit.service;

import by.bsuir.bank.common.api.BankException;
import by.bsuir.bank.common.client.ClientApi;
import by.bsuir.bank.common.client.ClientInfo;

```

```

import by.bsuir.bank.common.ledger.Batch;
import by.bsuir.bank.common.ledger.LedgerApi;
import by.bsuir.bank.common.ledger.OpenAccount;
import by.bsuir.bank.common.ledger.Posting;
import by.bsuir.bank.deposit.domain.DepositContract;
import by.bsuir.bank.deposit.domain.DepositKind;
import by.bsuir.bank.deposit.domain.DepositProduct;
import by.bsuir.bank.deposit.domain.DepositStatus;
import by.bsuir.bank.deposit.dto.DepositDetails;
import by.bsuir.bank.deposit.dto.DepositRequest;
import by.bsuir.bank.deposit.dto.DepositView;
import by.bsuir.bank.deposit.dto.Meta;
import by.bsuir.bank.deposit.repository.DepositContractRepository;
import by.bsuir.bank.deposit.repository.DepositProductRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.client.HttpClientErrorException;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

@Service
@RequiredArgsConstructor
public class DepositService {

    private static final DateTimeFormatter RU_DATE = DateTimeFormatter.ofPattern("dd.MM.yyyy");

    private final DepositContractRepository contracts;
    private final DepositProductRepository products;
    private final ClientApi clients;
    private final LedgerApi ledger;

    public Meta meta() {
        String last = contracts.maxNumber();
        int next = last == null ? 1 : Integer.parseInt(last.substring(2)) + 1;
        return new Meta(ledger.bankDay().date(), ledger.currencies(), String.format("Д-%06d", next));
    }

    @Transactional(readOnly = true)
    public List<DepositView> list(Long clientId) {
        List<DepositContract> found = clientId == null
            ? contracts.findAllByOrderByIdDesc()
            : contracts.findByIdOrderById(clientId);
        return found.stream().map(DepositView::from).toList();
    }

    @Transactional(readOnly = true)
    public DepositDetails details(Long id) {
        DepositContract contract = find(id);
        return new DepositDetails(DepositView.from(contract),
            ledger.accounts(contract.getNumber()), ledger.operations(contract.getNumber()));
    }

    /**
     * Заключение договора: проверка условий программы, открытие двух счетов (основной суммы и процентов)
     * и проводки приёма вклада. Запросы к главной книге идемпотентны по номеру договора,
     * поэтому повтор после сбоя не создаёт счета и проводки дважды.
     */
    @Transactional
    public DepositView open(DepositRequest request) {
        DepositProduct product = products.findById(request.productId())
            .orElseThrow(() -> BankException.fields(Map.of("productId", "Вид депозита отсутствует в справочнике")));
        LocalDate today = ledger.bankDay().date();
        checkTerms(request, product, today);
        if (contracts.existsByNumber(request.number())) {
            throw new BankException(HttpStatus.CONFLICT, "DUPLICATE_CONTRACT", "Договор уже существует",
                Map.of("number", "Договор с таким номером уже заключён"));
        }
        ClientInfo client = findClient(request.clientId());

        DepositKind kind = product.getKind();
        String number = request.number();
        String currency = product.getCurrency();
        String main = ledger.open(new OpenAccount(kind.getMainChartCode(), currency,
            client.id().intValue(), client.fullName(), number)).number();
    }

```

```

String interest = ledger.open(new OpenAccount(kind.getInterestChartCode(), currency,
    client.id().intValue(), client.fullName(), number)).number();
post(key(number, "OPEN"), number, today,
    DepositPostings.open(cash(currency), fund(currency), main, request.amount()));

DepositContract contract = new DepositContract();
contract.setNumber(number);
contract.setProduct(product);
contract.setClientId(client.id());
contract.setClientName(client.fullName());
contract.setCurrency(currency);
contract.setAmount(request.amount());
contract.setRate(product.getRate());
contract.setTermMonths(request.termMonths());
contract.setStartDate(today);
contract.setEndDate(request.endDate());
contract.setMainAccount(main);
contract.setInterestAccount(interest);
contract.setStatus(DepositStatus.ACTIVE);
return DepositView.from(contracts.save(contract));
}

/** Контроль корректности условий договора относительно выбранной депозитной программы. */
private void checkTerms(DepositRequest request, DepositProduct product, LocalDate today) {
    Map<String, String> errors = new LinkedHashMap<>();
    if (!product.getCurrency().equals(request.currency())) {
        errors.put("productId", "Программа недоступна в валюте " + request.currency());
    }
    if (request.amount().compareTo(product.getMinAmount()) < 0) {
        errors.put("amount", "Минимальная сумма вклада — " + product.getMinAmount() + " " + product.getCurrency());
    }
    if (request.rate().compareTo(product.getRate()) != 0) {
        errors.put("rate", "Ставка по программе — " + product.getRate() + " %");
    }
    if (request.termMonths() < product.getMinTermMonths() || request.termMonths() > product.getMaxTermMonths()) {
        errors.put("termMonths", "Срок по программе: от " + product.getMinTermMonths()
            + " до " + product.getMaxTermMonths() + " мес.");
    }
    if (!request.startDate().equals(today)) {
        errors.put("startDate", "Договор заключается текущим банковским днём: " + RU_DATE.format(today));
    }
    if (!request.endDate().equals(request.startDate().plusMonths(request.termMonths()))) {
        errors.put("endDate", "Дата окончания не соответствует сроку договора");
    }
    if (!errors.isEmpty()) {
        throw BankException.fields(errors);
    }
}

private ClientInfo findClient(Long clientId) {
    try {
        return clients.get(clientId);
    } catch (HttpClientErrorException.NotFound e) {
        throw BankException.fields(Map.of("clientId", "Клиент не найден в модуле «Клиенты»"));
    }
}

/** Досрочный отзыв вклада: доступен только для отзывных программ. */
@Transactional
public DepositView withdraw(Long id) {
    DepositContract contract = find(id);
    if (!contract.isActive()) {
        throw BankException.conflict("CONTRACT_CLOSED", "Договор " + contract.getNumber() + " уже закрыт");
    }
    if (contract.getProduct().getKind() == DepositKind.IRREVOCABLE) {
        throw BankException.conflict("IRREVOCABLE", "Безотзывный вклад нельзя забрать до окончания срока договора");
    }
    LocalDate today = ledger.bankDay().date();
    payInterest(contract, today);
    close(contract, today);
    return DepositView.from(contract);
}

/**
 * Обработка договора при открытии банковского дня date: начисление процентов за прошедшие дни,
 * выплата (ежемесячно либо в конце срока) и возврат вклада по окончании договора.
 * Обработка накопительная и идемпотентная: пропуск дня или повтор не искажают суммы.
 */
@Transactional
public List<String> processDay(Long id, LocalDate date) {
    DepositContract contract = find(id);
    List<String> events = new ArrayList<>();

```

```

        if (!contract.isActive() || !date.isAfter(contract.getStartDate())) {
            return events;
        }
        String number = contract.getNumber();
        String currency = contract.getCurrency();
        boolean matured = !date.isBefore(contract.getEndDate());
        LocalDate upTo = matured ? contract.getEndDate() : date;

        BigDecimal target = InterestCalculator.accrued(contract.getAmount(), contract.getRate(), contract.getStartDate(), upTo);
        BigDecimal delta = target.subtract(contract.getAccrued());
        if (delta.signum() > 0) {
            post(key(number, "ACCRUE-" + date), number, date,
                List.of(DepositPostings.accrual(fund(currency), contract.getInterestAccount(), delta)));
            contract.setAccrued(target);
            events.add(RU_DATE.format(date) + ": " + number + " – начислены проценты " + delta + " " + currency);
        }

        int months = InterestCalculator.fullMonths(contract.getStartDate(), upTo);
        boolean monthlyPayout = contract.getProduct().getKind() == DepositKind.REVOCABLE && months > contract.getPaidMonths();
        if (matured || monthlyPayout) {
            BigDecimal paid = payInterest(contract, date);
            contract.setPaidMonths(months);
            if (paid.signum() > 0) {
                events.add(RU_DATE.format(date) + ": " + number + " – выплачены проценты " + paid + " " + currency);
            }
        }
        if (matured) {
            close(contract, date);
            events.add(RU_DATE.format(date) + ": " + number + " – срок договора истёк, вклад " + contract.getAmount() + " " +
                currency + " возвращён");
        }
        return events;
    }

    private BigDecimal payInterest(DepositContract contract, LocalDate date) {
        BigDecimal due = contract.getAccrued().subtract(contract.getPaid());
        if (due.signum() > 0) {
            post(key(contract.getNumber(), "PAYOUT-" + date), contract.getNumber(), date,
                DepositPostings.payout(cash(contract.getCurrency()), contract.getInterestAccount(), due));
            contract.setPaid(contract.getAccrued());
        }
        return due;
    }

    private void close(DepositContract contract, LocalDate date) {
        String currency = contract.getCurrency();
        post(key(contract.getNumber(), "CLOSE"), contract.getNumber(), date,
            DepositPostings.close(cash(currency), fund(currency), contract.getMainAccount(), contract.getAmount()));
        contract.setStatus(DepositStatus.CLOSED);
        contract.setClosedOn(date);
    }

    private void post(String batchKey, String contractRef, LocalDate date, List<Posting> postings) {
        ledger.post(new Batch(batchKey, contractRef, date, postings));
    }

    private static String key(String number, String step) {
        return "DEP-" + number + "-" + step;
    }

    private String cash(String currency) {
        return ledger.systemAccount(LedgerApi.CASH, currency).number();
    }

    private String fund(String currency) {
        return ledger.systemAccount(LedgerApi.FUND, currency).number();
    }

    private DepositContract find(Long id) {
        return contracts.findById(id).orElseThrow(() -> BankException.notFound("Депозитный договор не найден"));
    }
}

```

Листинг А.81 – deposit- service/src/main/java/by/bsuir/bank/deposit/service/InterestCalculator.java

```

package by.bsuir.bank.deposit.service;

import java.math.BigDecimal;
import java.math.RoundingMode;

```

```

import java.time.LocalDate;
import java.time.temporal.ChronoUnit;

/** Расчёт процентов по вкладу: простые проценты, фактическое число дней, 365 дней в году. */
public final class InterestCalculator {

    private static final BigDecimal YEAR_BASIS = BigDecimal.valueOf(100L * 365);

    /**
     * Проценты, накопленные с даты начала по дату расчёта:  $S * r * \text{дни} / (100 * 365)$ .
     * Считается накопленная сумма целиком, а не прирост за день, — так копейки округления не теряются.
     */
    public static BigDecimal accrued(BigDecimal amount, BigDecimal rate, LocalDate start, LocalDate upTo) {
        long days = Math.max(0, ChronoUnit.DAYS.between(start, upTo));
        return amount.multiply(rate).multiply(BigDecimal.valueOf(days)).divide(YEAR_BASIS, 2, RoundingMode.HALF_UP);
    }

    /** Число полных месяцев договора, истёкших к дате расчёта. */
    public static int fullMonths(LocalDate start, LocalDate upTo) {
        int months = 0;
        while (!start.plusMonths(months + 1L).isAfter(upTo)) {
            months++;
        }
        return months;
    }

    private InterestCalculator() {
    }
}

```

Листинг А.82 – deposit- service/src/main/java/by/bsuir/bank/deposit/web/DayCloseController.java

```

package by.bsuir.bank.deposit.web;

import by.bsuir.bank.deposit.service.DayCloseProcessor;
import lombok.RequiredArgsConstructor;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

import java.time.LocalDate;
import java.util.List;

/** Внутренний REST для сервиса «Счета»: обработка открытия нового банковского дня. */
@RestController
@RequiredArgsConstructor
public class DayCloseController {

    private final DayCloseProcessor processor;

    public record DayOpened(LocalDate date) {
    }

    @PostMapping("/api/internal/day-close")
    public List<String> dayOpened(@RequestBody DayOpened event) {
        return processor.process(event.date());
    }
}

```

Листинг А.83 – deposit-service/src/main/java/by/bsuir/bank/deposit/web/DepositController.java

```

package by.bsuir.bank.deposit.web;

import by.bsuir.bank.deposit.dto.DepositDetails;
import by.bsuir.bank.deposit.dto.DepositRequest;
import by.bsuir.bank.deposit.dto.DepositView;
import by.bsuir.bank.deposit.service.DepositService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;

```

```

import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
@RequestMapping("/api/deposits")
@RequiredArgsConstructor
public class DepositController {

    private final DepositService service;

    @GetMapping
    public List<DepositView> list(@RequestParam(required = false) Long clientId) {
        return service.list(clientId);
    }

    @GetMapping("/{id}")
    public DepositDetails details(@PathVariable Long id) {
        return service.details(id);
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public DepositView open(@Valid @RequestBody DepositRequest request) {
        return service.open(request);
    }

    /** Досрочный отзыв вклада. */
    @PostMapping("/{id}/withdraw")
    public DepositView withdraw(@PathVariable Long id) {
        return service.withdraw(id);
    }
}

```

Листинг А.84 – deposit-service/src/main/java/by/bsuir/bank/deposit/web/ReferenceController.java

```

package by.bsuir.bank.deposit.web;

import by.bsuir.bank.common.client.ClientApi;
import by.bsuir.bank.common.client.ClientInfo;
import by.bsuir.bank.common.ledger.BankDayInfo;
import by.bsuir.bank.common.ledger.DayCloseInfo;
import by.bsuir.bank.common.ledger.LedgerApi;
import by.bsuir.bank.deposit.domain.DepositProduct;
import by.bsuir.bank.deposit.dto.Meta;
import by.bsuir.bank.deposit.repository.DepositProductRepository;
import by.bsuir.bank.deposit.service.DepositService;
import lombok.RequiredArgsConstructor;
import org.springframework.data.domain.Sort;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

/** Справочные данные формы договора. Клиенты и банковский день запрашиваются у смежных сервисов. */
@RestController
@RequestMapping("/api")
@RequiredArgsConstructor
public class ReferenceController {

    private final DepositProductRepository products;
    private final DepositService deposits;
    private final ClientApi clients;
    private final LedgerApi ledger;

    @GetMapping("/products")
    public List<DepositProduct> products() {
        return products.findAll(Sort.by("id"));
    }

    @GetMapping("/clients")
    public List<ClientInfo> clients() {
        return clients.list();
    }
}

```

```

    @GetMapping("/meta")
    public Meta meta() {
        return deposits.meta();
    }

    @GetMapping("/bank-day")
    public BankDayInfo bankDay() {
        return ledger.bankDay();
    }

    @PostMapping("/bank-day/close")
    public DayCloseInfo closeDay(@RequestParam(defaultValue = "1") int days) {
        return ledger.closeDay(days);
    }
}

```

Листинг A.85 – deposit-service/src/main/resources/static/index.html

```

<!doctype html>
<html lang="ru">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Депозиты – Банк «Решение»</title>
  <link rel="stylesheet" href="/ui/bank.css">
</head>
<body>
  <div id="app" v-cloak></div>

  <script src="/webjars/vue/dist/vue.global.prod.js"></script>
  <script src="/ui/bank.js"></script>
  <script src="/app.js"></script>
</body>
</html>

```

Листинг A.86 – deposit-service/src/main/resources/static/app.js

```

/* Web-клиент сервиса депозитов: список договоров, форма заключения договора, карточка договора. */

const REQUIRED = 'Обязательное поле';

/* Дата окончания: как LocalDate.plusMonths – день месяца сохраняется либо прижимается к концу месяца. */
function addMonths(iso, months) {
  const [year, month, day] = iso.split('-').map(Number);
  const first = new Date(year, month - 1 + months, 1);
  const lastDay = new Date(first.getFullYear(), first.getMonth() + 1, 0).getDate();
  const pad = n => String(n).padStart(2, '0');
  return first.getFullYear() + '-' + pad(first.getMonth() + 1) + '-' + pad(Math.min(day, lastDay));
}

const app = Vue.createApp({
  data() {
    return {
      view: 'list', // list | form | details
      deposits: [],
      products: [],
      clients: [],
      meta: { bankDate: null, currencies: [], nextNumber: '' },
      form: {},
      errors: {},
      formError: '',
      details: null,
      notice: '',
      error: '',
      events: [],
      saving: false,
    };
  },
  computed: {
    availableProducts() {
      return this.products.filter(p => p.currency === this.form.currency);
    },
    product() {
      return this.products.find(p => p.id === this.form.productId) || null;
    },
    endDate() {
      const months = Number(this.form.termMonths);
      return this.meta.bankDate && months > 0 && months <= 600 ? addMonths(this.meta.bankDate, months) : null;
    }
  }
});

```

```

    },
  },
  async mounted() {
    try {
      this.products = await Bank.api('GET', '/api/products');
      await this.reload();
    } catch (e) {
      this.error = e.message;
    }
  },
  methods: {
    async reload() {
      this.deposits = await Bank.api('GET', '/api/deposits');
    },
    async openForm() {
      this.error = '';
      try {
        this.meta = await Bank.api('GET', '/api/meta');
        this.clients = await Bank.api('GET', '/api/clients');
      } catch (e) {
        this.error = e.message;
        return;
      }
      this.form = { clientId: null, currency: 'BYN', productId: null, number: this.meta.nextNumber, amount: '', termMonths: '' };
      this.errors = {};
      this.formError = '';
      this.notice = '';
      this.view = 'form';
    },
    /* Первичный контроль корректности данных договора; сервис повторяет все проверки. */
    validate() {
      const errors = {};
      const form = this.form;
      if (!form.clientId) errors.clientId = REQUIRED;
      if (!form.productId) errors.productId = REQUIRED;
      if (!form.number) errors.number = REQUIRED;
      else if (!/^Д-\d{6}$/.test(form.number)) errors.number = 'Формат номера договора: Д-000001';
      const amount = String(form.amount).trim().replace(',', '.');
      if (!amount) errors.amount = REQUIRED;
      else if (!/^\\d{1,12}(\\.\\d{1,2})?$/.test(amount)) errors.amount = 'Денежная сумма: только цифры и не более 2 знаков после запятой';
      else if (this.product && Number(amount) < this.product.minAmount)
        errors.amount = 'Минимальная сумма вклада - ' + Bank.money(this.product.minAmount) + ' ' + this.product.currency;
      const term = String(form.termMonths).trim();
      if (!term) errors.termMonths = REQUIRED;
      else if (!/^\\d{1,3}$/.test(term)) errors.termMonths = 'Срок - целое число месяцев';
      else if (this.product && (Number(term) < this.product.minTermMonths || Number(term) > this.product.maxTermMonths))
        errors.termMonths = 'Срок по программе: от ' + this.product.minTermMonths + ' до ' + this.product.maxTermMonths + ' мес.';
      return errors;
    },
    async save() {
      this.errors = this.validate();
      this.formError = Object.keys(this.errors).length ? 'Форма заполнена с ошибками - исправьте отмеченные поля' : '';
      if (this.formError) return;
      this.saving = true;
      try {
        const created = await Bank.api('POST', '/api/deposits', {
          number: this.form.number,
          productId: this.form.productId,
          currency: this.form.currency,
          clientId: this.form.clientId,
          amount: Number(String(this.form.amount).trim().replace(',', '.')),
          rate: this.product.rate,
          termMonths: Number(this.form.termMonths),
          startDate: this.meta.bankDate,
          endDate: this.endDate,
        });
        await this.reload();
        await this.openDetails(created.id);
        this.notice = 'Договор ' + created.number + ' заключён, открыты счета ' + created.mainAccount + ' и ' + created.interestAccount;
      } catch (e) {
        this.errors = e.fields;
        this.formError = e.message;
      } finally {
        this.saving = false;
      }
    },
    async openDetails(id) {
      this.notice = '';
      this.error = '';
      try {

```

```

        this.details = await Bank.api('GET', '/api/deposits/' + id);
        this.view = 'details';
      } catch (e) {
        this.error = e.message;
      }
    },
    async withdraw() {
      const contract = this.details.contract;
      if (!confirm('Отозвать вклад по договору ' + contract.number + '?')) return;
      try {
        await Bank.api('POST', '/api/deposits/' + contract.id + '/withdraw');
        await this.reload();
        await this.openDetails(contract.id);
        this.notice = 'Вклад возвращён клиенту, договор закрыт';
      } catch (e) {
        this.error = e.message;
      }
    },
    async dayClosed(result) {
      this.events = result.events;
      this.meta.bankDate = result.bankDate;
      await this.reload();
      if (this.view === 'details') await this.openDetails(this.details.contract.id);
    },
    /* Возврат к списку: сообщения карточки договора больше не нужны. */
    toList() {
      this.notice = '';
      this.view = 'list';
    },
    money: Bank.money,
    date: Bank.date,
  },
  watch: {
    'form.currency'() {
      if (this.product && this.product.currency !== this.form.currency) this.form.productId = null;
    },
  },
  template: `
<bank-nav active="deposits"></bank-nav>
<main>
  <div class="toolbar">
    <div>
      <h1>Депозитные договоры</h1>
      <p class="subtitle">Вклады физических лиц: заключение договора, начисление и выплата процентов</p>
    </div>
    <bank-day @changed="dayClosed" @failed="e => error = e.message"></bank-day>
  </div>

  <div v-if="error" id="error" class="alert error">{{ error }}</div>
  <div v-if="notice" id="notice" class="alert success">{{ notice }}</div>
  <div v-if="events.length" id="events" class="alert info">
    <b>Протокол закрытия дня</b>
    <div v-for="(event, i) in events.slice(-8)" :key="i">{{ event }}</div>
    <div v-if="events.length > 8" class="muted">... всего записей: {{ events.length }}</div>
  </div>

  <section v-if="view === 'list'">
    <div class="toolbar">
      <span class="muted">Договоров: {{ deposits.length }}</span>
      <button id="btn-add" class="primary" @click="openForm">Заключить договор</button>
    </div>
    <div class="card">
      <table id="deposits">
        <thead>
          <tr>
            <th>Договор</th>
            <th>Клиент</th>
            <th>Вид депозита</th>
            <th class="num">Сумма</th>
            <th class="num">Ставка</th>
            <th>Начало</th>
            <th>Окончание</th>
            <th class="num">Начислено</th>
            <th class="num">Выплачено</th>
            <th>Статус</th>
          </tr>
        </thead>
        <tbody>
          <tr v-for="d in deposits" :key="d.id">
            <td class="mono"><a class="link" @click="openDetails(d.id)">{{ d.number }}</a></td>
            <td>{{ d.clientName }}</td>
            <td>{{ d.productName }}<div class="muted" style="font-size: 12.5px">{{ d.kindTitle }}</div></td>
            <td class="num">{{ money(d.amount) }} {{ d.currency }}</td>
            <td class="num">{{ d.rate }} %</td>
            <td>{{ date(d.startDate) }}</td>
            <td>{{ date(d.endDate) }}</td>
            <td class="num">{{ money(d.accrued) }}</td>
            <td class="num">{{ money(d.paid) }}</td>
            <td><span class="badge" :class="d.status === 'ACTIVE' ? 'ok' : 'off'">{{ d.status === 'ACTIVE' ? 'Действует' : 'Закрыт' }}</span></td>
          </tr>
        </tbody>
      </table>
    </div>
  </section>
`

```

```

        </tr>
      </tbody>
    </table>
    <div v-if="!deposits.length" class="empty">Договоров пока нет</div>
  </div>
</section>

<section v-if="view === 'form'">
  <h2>Новый депозитный договор</h2>
  <div v-if="formError" id="form-alert" class="alert error">{{ formError }}</div>
  <form class="card" novalidate @submit.prevent="save">
    <div class="form-grid">
      <div class="field wide" :class="{ invalid: errors.clientId }">
        <label for="f-clientId">Клиент <span class="req">*</span></label>
        <select id="f-clientId" v-model="form.clientId">
          <option value="null">— выберите клиента —</option>
          <option v-for="c in clients" :key="c.id" :value="c.id">
            {{ c.fullName }} (паспорт {{ c.passportSeries }} {{ c.passportNumber }})
          </option>
        </select>
        <div v-if="errors.clientId" class="error" id="e-clientId">{{ errors.clientId }}</div>
      </div>
      <div class="field" :class="{ invalid: errors.number }">
        <label for="f-number">Номер договора <span class="req">*</span></label>
        <input id="f-number" type="text" v-model="form.number" v-mask="'Д-####'" placeholder="Д-000001">
        <div v-if="errors.number" class="error" id="e-number">{{ errors.number }}</div>
      </div>

      <div class="field">
        <label for="f-currency">Валюта <span class="req">*</span></label>
        <select id="f-currency" v-model="form.currency">
          <option v-for="c in meta.currencies" :key="c.code" :value="c.code">{{ c.code }} — {{ c.name }}</option>
        </select>
      </div>
      <div class="field wide" :class="{ invalid: errors.productId }">
        <label for="f-productId">Вид депозита <span class="req">*</span></label>
        <select id="f-productId" v-model="form.productId">
          <option value="null">— выберите программу —</option>
          <option v-for="p in availableProducts" :key="p.id" :value="p.id">
            «{{ p.name }}» — {{ p.kindTitle.toLowerCase() }}, {{ p.rate }} % годовых
          </option>
        </select>
        <div v-if="errors.productId" class="error" id="e-productId">{{ errors.productId }}</div>
        <div v-else-if="product" class="hint">{{ product.description }}. Сумма от {{ money(product.minAmount) }} {{ product.
currency }},
        срок {{ product.minTermMonths }}—{{ product.maxTermMonths }} мес.</div>
      </div>

      <div class="field" :class="{ invalid: errors.amount }">
        <label for="f-amount">Сумма вклада, {{ form.currency }} <span class="req">*</span></label>
        <input id="f-amount" type="text" v-model="form.amount" placeholder="0.00">
        <div v-if="errors.amount" class="error" id="e-amount">{{ errors.amount }}</div>
      </div>
      <div class="field" :class="{ invalid: errors.termMonths }">
        <label for="f-termMonths">Срок договора, мес. <span class="req">*</span></label>
        <input id="f-termMonths" type="text" v-model="form.termMonths" v-mask="###">
        <div v-if="errors.termMonths" class="error" id="e-termMonths">{{ errors.termMonths }}</div>
      </div>
      <div class="field" :class="{ invalid: errors.rate }">
        <label for="f-rate">Процент по вкладу, % годовых</label>
        <input id="f-rate" type="text" :value="product ? product.rate : ''" readonly>
        <div v-if="errors.rate" class="error" id="e-rate">{{ errors.rate }}</div>
      </div>

      <div class="field" :class="{ invalid: errors.startDate }">
        <label for="f-startDate">Дата начала</label>
        <input id="f-startDate" type="text" :value="date(meta.bankDate)" readonly>
        <div v-if="errors.startDate" class="error" id="e-startDate">{{ errors.startDate }}</div>
      </div>
      <div class="field" :class="{ invalid: errors.endDate }">
        <label for="f-endDate">Дата окончания</label>
        <input id="f-endDate" type="text" :value="date(endDate)" readonly>
        <div v-if="errors.endDate" class="error" id="e-endDate">{{ errors.endDate }}</div>
      </div>
    </div>
    <div class="form-actions" style="margin-top: 18px">
      <button id="btn-save" type="submit" class="primary" :disabled="saving">Заклучить договор</button>
      <button id="btn-cancel" type="button" @click="view = 'list'">Отмена</button>
    </div>
  </form>
</section>

```

```

<section v-if="view === 'details' && details">
  <div class="toolbar">
    <h2>Дорогов {{ details.contract.number }} · {{ details.contract.clientName }}</h2>
    <div style="display: flex; gap: 8px">
      <button v-if="details.contract.status === 'ACTIVE' && details.contract.kind === 'REVOCABLE'"
        id="btn-withdraw" class="danger" @click="withdraw">Отозвать вклад</button>
      <button id="btn-back" @click="toList">К списку</button>
    </div>
  </div>
  <div class="card details">
    <div><div class="label">Вид депозита</div><div class="value">«{{ details.contract.productName }}»</div><div class="muted"
">{{ details.contract.kindTitle }}</div></div>
    <div><div class="label">Сумма вклада</div><div class="value">{{ money(details.contract.amount) }} {{ details.contract.
currency }}</div></div>
    <div><div class="label">Ставка</div><div class="value">{{ details.contract.rate }} % годовых</div></div>
    <div><div class="label">Статус</div><div class="value">{{ details.contract.status === 'ACTIVE' ? 'Действует' : 'Закрыт '
+ date(details.contract.closedOn) }}</div></div>
    <div><div class="label">Срок</div><div class="value">{{ details.contract.termMonths }} мес.</div></div>
    <div><div class="label">Период</div><div class="value">{{ date(details.contract.startDate) }} – {{ date(details.contract
.endDate) }}</div></div>
    <div><div class="label">Начислено процентов</div><div class="value">{{ money(details.contract.accrued) }} {{ details.
contract.currency }}</div></div>
    <div><div class="label">Выплачено процентов</div><div class="value">{{ money(details.contract.paid) }} {{ details.
contract.currency }}</div></div>
  </div>

  <div class="card">
    <h2>Счета договора</h2>
    <table id="contract-accounts">
      <thead><tr><th>Номер счёта</th><th>Код</th><th>Назначение</th><th>Активность</th><th class="num">Дебет</th><th class="
num">Кредит</th><th class="num">Сальдо</th></tr></thead>
      <tbody>
        <tr v-for="a in details.accounts" :key="a.number">
          <td class="mono">{{ a.number }}</td>
          <td class="mono">{{ a.chartCode }}</td>
          <td>{{ a.chartName }}<div class="muted" style="font-size: 12.5px">{{ a.name }}</div></td>
          <td>{{ a.activityTitle }}</td>
          <td class="num">{{ money(a.debit) }}</td>
          <td class="num">{{ money(a.credit) }}</td>
          <td class="num"><b>{{ money(a.balance) }}</b></td>
        </tr>
      </tbody>
    </table>
  </div>

  <div class="card">
    <h2>Проводки по договору</h2>
    <table id="contract-operations">
      <thead><tr><th>Дата</th><th>Операция</th><th>Счёт</th><th class="num">Дебет</th><th class="num">Кредит</th></tr></thead>
      <tbody>
        <template v-for="op in details.operations" :key="op.id">
          <tr v-for="(entry, i) in op.entries" :key="op.id + '-' + i">
            <td>{{ i === 0 ? date(op.bankDate) : '' }}</td>
            <td>{{ i === 0 ? op.description : '' }}</td>
            <td class="mono">{{ entry.account }} <span class="muted">{{ entry.chartCode }}</span></td>
            <td class="num">{{ entry.side === 'DEBIT' ? money(entry.amount) : '' }}</td>
            <td class="num">{{ entry.side === 'CREDIT' ? money(entry.amount) : '' }}</td>
          </tr>
        </template>
      </tbody>
    </table>
  </div>
</section>
</main>',
});

app.component('bank-nav', Bank.navComponent);
app.component('bank-day', Bank.dayComponent);
app.directive('mask', Bank.maskDirective);
app.mount('#app');

```

Листинг А.87 – deposit-service/src/test/java/by/bsuir/bank/deposit/DepositApiIT.java

```

package by.bsuir.bank.deposit;

import by.bsuir.bank.common.client.ClientApi;
import by.bsuir.bank.common.client.ClientInfo;
import by.bsuir.bank.common.ledger.InMemoryLedger;

```

```

import by.bsuir.bank.deposit.repository.DepositContractRepository;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import org.junit.jupiter.params.provider.ValueSource;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.context.annotation.Import;
import org.springframework.http.HttpStatus;
import org.springframework.http.MediaType;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.context.bean.override.mockito.MockitoSpyBean;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.ResultActions;
import org.springframework.web.client.HttpClientErrorException;
import org.springframework.web.client.ResourceAccessException;

import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

import static org.assertj.core.api.Assertions.assertThat;
import static org.hamcrest.Matchers.hasSize;
import static org.mockito.Mockito.doThrow;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

/** Интеграционные тесты REST сервиса депозитов: контроль корректности данных договора и ошибки смежных сервисов. */
@SpringBootTest(properties = "spring.datasource.url=jdbc:h2:mem:${random.uuid};DB_CLOSE_DELAY=-1")
@AutoConfigureMockMvc
@Import({TestLedgerConfig.class})
class DepositApiIT {

    @Autowired
    private MockMvc mvc;
    @Autowired
    private ObjectMapper json;
    @Autowired
    private DepositContractRepository contracts;
    @MockitoSpyBean
    private InMemoryLedger ledger;
    @MockitoBean
    private ClientApi clients;

    @BeforeEach
    void cleanState() {
        contracts.deleteAll();
        ledger.reset();
        ClientInfo client = new ClientInfo(1L, "Иванов Иван Иванович", "MP", "7654321", "3170590A077PB4");
        when(clients.get(1L)).thenReturn(client);
        when(clients.list()).thenReturn(List.of(client));
    }

    @Test
    @DisplayName("Справочники формы: виды депозита, валюты, клиенты, следующий номер договора")
    void references() throws Exception {
        mvc.perform(get("/api/products")).andExpect(jsonPath("$", hasSize(5)))
            .andExpect(jsonPath("$[0].kindTitle").value("Отзывный, до востребования"));
        mvc.perform(get("/api/clients")).andExpect(jsonPath("$[0].fullName").value("Иванов Иван Иванович"));
        mvc.perform(get("/api/meta"))
            .andExpect(jsonPath("$.bankDate").value("2026-10-01"))
            .andExpect(jsonPath("$.currencies", hasSize(3)))
            .andExpect(jsonPath("$.nextNumber").value("Д-000001"));
    }

    @Test
    @DisplayName("Договор заключается и появляется в списке, номер следующего договора увеличивается")
    void createsContract() throws Exception {
        create(valid()).andExpect(status().isCreated())
            .andExpect(jsonPath("$.number").value("Д-000001"))
            .andExpect(jsonPath("$.clientName").value("Иванов Иван Иванович"))
            .andExpect(jsonPath("$.mainAccount").isString())
            .andExpect(jsonPath("$.interestAccount").isString());

        mvc.perform(get("/api/deposits")).andExpect(jsonPath("$", hasSize(1)));
    }
}

```

```

        mvc.perform(get("/api/deposits/" + contracts.findAll().get(0).getId()))
            .andExpect(jsonPath("$.accounts", hasSize(2)))
            .andExpect(jsonPath("$.operations", hasSize(3)));
        mvc.perform(get("/api/meta")).andExpect(jsonPath("$.nextNumber").value("Д-000002"));
    }

    @ParameterizedTest(name = "без поля {0}")
    @ValueSource(strings = {"number", "productId", "currency", "clientId", "amount", "rate", "termMonths", "startDate", "endDate"})
    @DisplayName("Все поля договора обязательны")
    void requiredFields(String field) throws Exception {
        Map<String, Object> request = valid();
        request.remove(field);

        create(request).andExpect(status().isBadRequest())
            .andExpect(jsonPath("$.fields." + field).value("Обязательное поле"));
        assertThat(contracts.count()).isZero();
    }

    @ParameterizedTest(name = "{0} = {1}")
    @CsvSource({
        "number, 000001",
        "number, D-1",
        "amount, 99.99",
        "amount, -5",
        "amount, 100.001",
        "rate, 8.50",
        "termMonths, 0",
        "termMonths, 13",
        "startDate, 2026-10-02",
        "endDate, 2027-01-02",
        "currency, USD",
        "productId, 77",
    })
    @DisplayName("Условия договора сверяются с депозитной программой и банковским днём")
    void termsAreChecked(String field, String value) throws Exception {
        Map<String, Object> request = valid();
        request.put(field, value);

        String errorField = "currency".equals(field) ? "productId" : field;
        create(request).andExpect(status().isBadRequest())
            .andExpect(jsonPath("$.fields." + errorField).exists());
        assertThat(contracts.count()).isZero();
        assertThat(ledger.journal()).isEmpty();
    }

    @Test
    @DisplayName("Номер договора уникален")
    void duplicateNumber() throws Exception {
        create(valid()).andExpect(status().isCreated());

        create(valid()).andExpect(status().isConflict())
            .andExpect(jsonPath("$.fields.number").exists());
        assertThat(contracts.count()).isEqualTo(1);
    }

    @Test
    @DisplayName("Клиент должен существовать в модуле «Клиенты»")
    void unknownClient() throws Exception {
        when(clients.get(99L)).thenReturn(HttpClientErrorException.create(HttpStatus.NOT_FOUND, "Not Found", null, null, null));
        Map<String, Object> request = valid();
        request.put("clientId", 99);

        create(request).andExpect(status().isBadRequest())
            .andExpect(jsonPath("$.fields.clientId").value("Клиент не найден в модуле «Клиенты»"));
        assertThat(ledger.journal()).isEmpty();
    }

    @Test
    @DisplayName("Сервис «Счета» недоступен – ответ 503, договор не создаётся")
    void ledgerIsDown() throws Exception {
        doThrow(new ResourceAccessException("Connection refused")).when(ledger).bankDay();

        create(valid()).andExpect(status().isServiceUnavailable())
            .andExpect(jsonPath("$.code").value("SERVICE_UNAVAILABLE"));
        assertThat(contracts.count()).isZero();
    }

    @Test
    @DisplayName("Внутренний REST закрытия дня возвращает протокол начислений")
    void dayCloseEndpoint() throws Exception {
        create(valid()).andExpect(status().isCreated());
    }

```

```

        mvc.perform(post("/api/internal/day-close").contentType(MediaType.APPLICATION_JSON).content("{\"date\": \"2026-10-11\"}"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$", hasSize(1)))
            .andExpect(jsonPath("$.value").value("11.10.2026: Д-000001 – начислены проценты 1.92 BYN"));
    }

    @Test
    @DisplayName("Безотзывный вклад нельзя отозвать через REST")
    void irrevocableWithdrawIsRejected() throws Exception {
        Map<String, Object> request = valid();
        request.put("productId", 2);
        request.put("amount", 500);
        request.put("rate", 12.90);
        create(request).andExpect(status().isCreated());

        mvc.perform(post("/api/deposits/" + contracts.findAll().get(0).getId() + "/withdraw"))
            .andExpect(status().isConflict())
            .andExpect(jsonPath("$.code").value("IRREVOCABLE"));
    }

    private Map<String, Object> valid() {
        Map<String, Object> request = new LinkedHashMap<>();
        request.put("number", "Д-000001");
        request.put("productId", 1);
        request.put("currency", "BYN");
        request.put("clientId", 1);
        request.put("amount", 1000);
        request.put("rate", 7.00);
        request.put("termMonths", 3);
        request.put("startDate", "2026-10-01");
        request.put("endDate", "2027-01-01");
        return request;
    }

    private ResultActions create(Map<String, Object> request) throws Exception {
        return mvc.perform(post("/api/deposits").contentType(MediaType.APPLICATION_JSON)
            .content(json.writeValueAsString(request)));
    }
}

```

Листинг А.88 –

deposit-service/src/test/java/by/bsuir/bank/deposit/DepositFlowIT.java

```

package by.bsuir.bank.deposit;

import by.bsuir.bank.common.api.BankException;
import by.bsuir.bank.common.client.ClientApi;
import by.bsuir.bank.common.client.ClientInfo;
import by.bsuir.bank.common.ledger.AccountInfo;
import by.bsuir.bank.common.ledger.InMemoryLedger;
import by.bsuir.bank.common.ledger.LedgerApi;
import by.bsuir.bank.common.ledger.OperationInfo;
import by.bsuir.bank.deposit.dto.DepositRequest;
import by.bsuir.bank.deposit.dto.DepositView;
import by.bsuir.bank.deposit.repository.DepositContractRepository;
import by.bsuir.bank.deposit.service.DayCloseProcessor;
import by.bsuir.bank.deposit.service.DepositService;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.context.annotation.Import;
import org.springframework.test.context.bean.override.mockito.MockitoBean;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;

import static by.bsuir.bank.common.ledger.InMemoryLedger.START;
import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;
import static org.mockito.ArgumentMatchers.anyLong;
import static org.mockito.Mockito.when;

/**
 * Интеграционные тесты депозитного процесса: сервис, база H2 и главная книга в памяти.
 * Обороты счетов сверяются со схемой проводок «Депозитная программа» из задания.
 */

```

```

@SpringBootTest(properties = "spring.datasource.url=jdbc:h2:mem:${random.uuid};DB_CLOSE_DELAY=-1")
@Import(TestLedgerConfig.class)
class DepositFlowIT {

    private static final long REVOCABLE_BYN = 1;        // «Свободное накопление», 7 %, проценты ежемесячно
    private static final long IRREVOCABLE_BYN = 2;      // «Online-решение», 12,9 %, проценты в конце срока

    @Autowired
    private DepositService deposits;
    @Autowired
    private DayCloseProcessor dayClose;
    @Autowired
    private DepositContractRepository contracts;
    @Autowired
    private InMemoryLedger ledger;
    @MockitoBean
    private ClientApi clients;

    private String cash;
    private String fund;

    @BeforeEach
    void cleanState() {
        contracts.deleteAll();
        ledger.reset();
        cash = ledger.systemAccount(LedgerApi.CASH, "BYN").number();
        fund = ledger.systemAccount(LedgerApi.FUND, "BYN").number();
        when(clients.get(anyLong())).thenAnswer(call ->
            new ClientInfo(call.getArgument(0), "Иванов Иван Иванович", "MP", "7654321", "3170590A077PB4"));
    }

    @Test
    @DisplayName("Заключение договора: два счёта по плану счетов и три проводки приёма вклада")
    void openingContract() {
        DepositView deposit = open("Д-000001", REVOCABLE_BYN, "1000.00", 3);

        List<AccountInfo> accounts = ledger.accounts("Д-000001");
        assertThat(accounts).extracting(AccountInfo::chartCode).containsExactly("3404", "3470");
        assertThat(accounts).allSatisfy(a -> {
            assertThat(a.activity()).isEqualTo("PASSIVE");
            assertThat(a.name()).isEqualTo("Иванов Иван Иванович");
        });
        assertThat(ledger.operations("Д-000001")).extracting(OperationInfo::description).containsExactly(
            "Внесение денег в кассу", "Перевод денег с кассы на текущий счёт", "Использование денег банком");

        // касса: дебет +1000 и кредит -1000; счёт вклада: кредит +1000 и дебет -1000; фонд: кредит +1000
        assertTurnover(cash, "1000.00", "1000.00", "0.00");
        assertTurnover(deposit.mainAccount(), "1000.00", "1000.00", "0.00");
        assertThat(ledger.balance(fund)).isEqualToByComparingTo(InMemoryLedger.CAPITAL.add(new BigDecimal("1000.00")));
        assertThat(deposit.status()).isEqualTo("ACTIVE");
        assertThat(deposit.endDate()).isEqualTo(START.plusMonths(3));
    }

    @Test
    @DisplayName("Отзывный вклад: проценты начисляются ежедневно и выплачиваются раз в месяц")
    void revocableDepositPaysInterestMonthly() {
        DepositView deposit = open("Д-000001", REVOCABLE_BYN, "1000.00", 3);

        closeDays(30); // 31.10: начислено за 30 дней, выплаты ещё нет
        assertTurnover(deposit.interestAccount(), "0.00", "5.75", "5.75");
        assertThat(view(deposit).paid()).isEqualToByComparingTo("0");

        closeDays(1); // 01.11: прошёл месяц — выплата 5,95
        assertTurnover(deposit.interestAccount(), "5.95", "5.95", "0.00");
        assertTurnover(cash, "1005.95", "1005.95", "0.00");
        assertThat(view(deposit).accrued()).isEqualToByComparingTo("5.95");
        assertThat(view(deposit).paid()).isEqualToByComparingTo("5.95");
        assertThat(view(deposit).status()).isEqualTo("ACTIVE");
    }

    @Test
    @DisplayName("Срочный вклад: проценты выплачиваются в конце срока вместе с возвратом суммы")
    void irrevocableDepositPaysAtMaturity() {
        DepositView deposit = open("Д-000002", IRREVOCABLE_BYN, "5000.00", 3);

        closeDays(91); // 31.12: до окончания срока один день
        assertThat(view(deposit).paid()).isEqualToByComparingTo("0");
        assertThat(ledger.balance(deposit.interestAccount())).isEqualToByComparingTo("160.81");

        List<String> events = closeDays(1); // 01.01.2027: окончание договора, 92 дня
        assertThat(events).anyMatch(e -> e.contains("срок договора истёк"));
        DepositView closed = view(deposit);
    }
}

```

```

assertThat(closed.status()).isEqualTo("CLOSED");
assertThat(closed.closedOn()).isEqualTo(LocalDate.of(2027, 1, 1));
assertThat(closed.accrued()).isEqualByComparingTo("162.58");
assertThat(closed.paid()).isEqualByComparingTo("162.58");

// все счета клиента обнулились, касса транзитом пропустила вклад и проценты, фонд потратил проценты
assertTurnover(deposit.mainAccount(), "10000.00", "10000.00", "0.00");
assertTurnover(deposit.interestAccount(), "162.58", "162.58", "0.00");
assertTurnover(cash, "10162.58", "10162.58", "0.00");
assertThat(ledger.balance(fund)).isEqualByComparingTo(InMemoryLedger.CAPITAL.subtract(new BigDecimal("162.58")));
assertThat(ledger.operations("Д-000002")).extracting(OperationInfo::description).contains(
    "Перевод процентов в кассу", "Вывод процентов из кассы",
    "Окончание депозита", "Перевод депозита в кассу", "Вывод денег из кассы");

assertThat(closeDays(5)).isEmpty(); // закрытый договор больше не обрабатывается
}

@Test
@DisplayName("Повторная обработка того же дня не начисляет проценты второй раз")
void dayProcessingIsIdempotent() {
    DepositView deposit = open("Д-000001", REVOCABLE_BYN, "1000.00", 3);
    LocalDate date = START.plusDays(10);

    dayClose.process(date);
    BigDecimal accrued = ledger.credit(deposit.interestAccount());
    assertThat(dayClose.process(date)).isEmpty();

    assertThat(accrued).isEqualByComparingTo("1.92");
    assertThat(ledger.credit(deposit.interestAccount())).isEqualByComparingTo(accrued);
}

@Test
@DisplayName("Пропущенные дни навёрстываются при следующей обработке")
void skippedDaysAreCaughtUp() {
    DepositView deposit = open("Д-000001", REVOCABLE_BYN, "1000.00", 3);

    dayClose.process(START.plusDays(45)); // сервис «проспал» 44 дня

    assertThat(view(deposit).accrued()).isEqualByComparingTo("8.63");
    assertThat(view(deposit).paid()).isEqualByComparingTo("8.63");
}

@Test
@DisplayName("Досрочный отзыв: отзывный вклад возвращается с процентами, безотзывный – отказ")
void earlyWithdrawal() {
    DepositView revocable = open("Д-000001", REVOCABLE_BYN, "1000.00", 6);
    DepositView irrevocable = open("Д-000002", IRREVOCABLE_BYN, "5000.00", 6);
    closeDays(10);

    DepositView closed = deposits.withdraw(revocable.id());
    assertThat(closed.status()).isEqualTo("CLOSED");
    assertThat(closed.paid()).isEqualByComparingTo("1.92");
    assertThat(ledger.balance(revocable.mainAccount())).isEqualByComparingTo("0");
    assertThat(ledger.balance(revocable.interestAccount())).isEqualByComparingTo("0");

    assertThatThrownBy(() -> deposits.withdraw(irrevocable.id()))
        .isInstanceOfSatisfying(BankException.class, e -> assertThat(e.getCode()).isEqualTo("IRREVOCABLE"));
    assertThatThrownBy(() -> deposits.withdraw(revocable.id()))
        .isInstanceOfSatisfying(BankException.class, e -> assertThat(e.getCode()).isEqualTo("CONTRACT_CLOSED"));
    assertThat(view(irrevocable).status()).isEqualTo("ACTIVE");
}

@Test
@DisplayName("При двух активных депозитных программах в отчёте не менее шести счетов")
void twoProgramsGiveSixAccounts() {
    DepositView first = open("Д-000001", REVOCABLE_BYN, "1000.00", 3);
    DepositView second = open("Д-000002", IRREVOCABLE_BYN, "5000.00", 3);

    assertThat(List.of(cash, fund, first.mainAccount(), first.interestAccount(),
        second.mainAccount(), second.interestAccount())).doesNotHaveDuplicates().hasSize(6);
    assertThat(ledger.accounts("Д-000002")).extracting(AccountInfo::chartCode).containsExactly("3414", "3471");
}

private DepositView open(String number, long productId, String amount, int months) {
    return deposits.open(new DepositRequest(number, productId, "BYN", 1L, new BigDecimal(amount),
        productId == REVOCABLE_BYN ? new BigDecimal("7.00") : new BigDecimal("12.90"),
        months, START, START.plusMonths(months)));
}

/** Закрытие дней подряд – то, что делает сервис «Счета», вызывая участников. */
private List<String> closeDays(int days) {
    List<String> events = new java.util.ArrayList<>();

```

```

        for (int i = 0; i < days; i++) {
            LocalDate next = ledger.bankDay().date().plusDays(1);
            events.addAll(dayClose.process(next));
            ledger.setDate(next);
        }
        return events;
    }

    private DepositView view(DepositView deposit) {
        return deposits.details(deposit.id()).contract();
    }

    private void assertTurnover(String account, String debit, String credit, String balance) {
        assertThat(ledger.debit(account)).as("дебет %s", account).isEqualToComparingTo(debit);
        assertThat(ledger.credit(account)).as("кредит %s", account).isEqualToComparingTo(credit);
        assertThat(ledger.balance(account)).as("сальдо %s", account).isEqualToComparingTo(balance);
    }
}

```

Листинг А.89 –

deposit-service/src/test/java/by/bsuir/bank/deposit/InterestCalculatorTest.java

```

package by.bsuir.bank.deposit;

import by.bsuir.bank.deposit.service.InterestCalculator;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;

import java.math.BigDecimal;
import java.time.LocalDate;

import static org.assertj.core.api.Assertions.assertThat;

/** Модульные тесты расчёта процентов по вкладу. */
class InterestCalculatorTest {

    private static final LocalDate START = LocalDate.of(2026, 10, 1);

    @ParameterizedTest(name = "{0} под {1} % за {2} дн. = {3}")
    @CsvSource({
        "1000.00, 7.00, 31, 5.95",
        "1000.00, 7.00, 1, 0.19",
        "5000.00, 12.90, 365, 645.00",
        "5000.00, 12.90, 396, 699.78",
        "100.00, 1.50, 0, 0.00",
    })
    void simpleInterestOnActualDays(String amount, String rate, int days, String expected) {
        BigDecimal accrued = InterestCalculator.accrued(new BigDecimal(amount), new BigDecimal(rate), START, START.plusDays(days));

        assertThat(accrued).isEqualToComparingTo(expected);
    }

    @Test
    void nothingIsAccruedBeforeStart() {
        assertThat(InterestCalculator.accrued(new BigDecimal("1000"), new BigDecimal("7"), START, START.minusDays(5)))
            .isEqualToComparingTo("0");
    }

    @Test
    void dailyIncrementsAddUpToTotalWithoutRoundingLoss() {
        BigDecimal amount = new BigDecimal("1234.56");
        BigDecimal rate = new BigDecimal("7.00");
        BigDecimal posted = BigDecimal.ZERO;
        for (int day = 1; day <= 92; day++) {
            BigDecimal target = InterestCalculator.accrued(amount, rate, START, START.plusDays(day));
            assertThat(target).isGreaterThanOrEqualTo(posted);
            posted = target;
        }
        assertThat(posted).isEqualToComparingTo(InterestCalculator.accrued(amount, rate, START, START.plusDays(92)));
        assertThat(posted).isEqualToComparingTo("21.78");
    }

    @ParameterizedTest(name = "с {0} по {1} – полных месяцев: {2}")
    @CsvSource({
        "2026-10-01, 2026-10-31, 0",
        "2026-10-01, 2026-11-01, 1",
        "2026-10-01, 2027-01-01, 3",
        "2026-01-31, 2026-02-27, 0",
    })

```

```

        "2026-01-31, 2026-02-28, 1",
        "2026-10-01, 2026-10-01, 0",
    })
    void fullMonthsFollowCalendar(LocalDate start, LocalDate upTo, int months) {
        assertThat(InterestCalculator.fullMonths(start, upTo)).isEqualTo(months);
    }
}

```

Листинг А.90 – deposit-service/src/test/java/by/bsuir/bank/deposit/TestLedgerConfig.java

```

package by.bsuir.bank.deposit;

import by.bsuir.bank.common.ledger.InMemoryLedger;
import org.springframework.boot.test.context.TestConfiguration;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Primary;

/** В тестах вместо REST-клиента сервиса «Счета» подставляется главная книга в памяти. */
@TestConfiguration
public class TestLedgerConfig {

    @Bean
    @Primary
    public InMemoryLedger inMemoryLedger() {
        return new InMemoryLedger();
    }
}

```